

Data Structures and Algorithms

Instructor: Dr. Hao Qin (秦浩)

E-mail: hao.qin@scupi.cn

My Office: Room 514, SCUPI (North)

Outline

- Part 1: Basic C++ Concepts
 - Constructors
 - Destructors
 - Class Composition
 - Composition with Initializer Lists
- Part 2: On-your-own Study Topics
 - Learning how to use the Visual C++/Xcode debuggers
 - A few topics you need to know to do real projects

Constructors and Destructors

What's the big picture?

Constructors

A **constructor function** is used to **reset an object's member variables** when the object is first created; otherwise they'd be random!

```
class SomeClass {  
public:  
    void blah() { cout << v; } // random value!  
private:  
    int v; // uninitialized if no constructor!  
};
```



Destructors

An object will often reserve memory slots from the operating system while it runs.

A **destructor function** guarantees **that reserved memory is freed** when an object goes away.

If you don't free this memory, your program will eventually run out of memory and crash.

Uses:

Constructors and destructors are required for virtually every correct C++ class.

Constructor Basics



Has 2
Params!

```
class Gassy
{
public:
    Gassy(int age, bool ateBeans)
    {
        m_age      = age;
        m_ateBeans = ateBeans;
    }

    int getFartsPerHr()
    {
        if(m_ateBeans == true)
            return(100);
        return(3 * m_age);
    }

private:
    int m_age;
    bool m_ateBeans;
};
```

Here's our Gassy class....
It represents gassy people.

Our Gassy class has a
constructor that takes two
parameters to initialize it.

If a constructor has
parameters, then you must
pass values in to them!

Requires
two
args!

```
int main()
{
    Gassy betty(18, true);
    Gassy alan; // error!
}
```

ctor Basics

```
class Gassy
{
public:
```

```
    Gassy(int age, bool ateBeans)
```

```
    {
```

```
        m_age = age;
```

```
        m_ateBeans = ateBe
```

```
    }
```

```
    int getFarts
```

```
    {
```

```
        if(m_ateBeans == true)
```

```
            return(100);
```

```
        return(3 * m_age);
```

```
    }
```

```
private:
```

```
    int m_age;
```

```
    bool m_ateBeans;
```

```
};
```

goes to this constructor, since it has matching parameters.

goes to this constructor, since it requires no parameters.

Your class can have many different constructors...

The only requirement is that each one **has different parameters** and/or types.

This is called **constructor overloading**.

This variable

and this initialization...

```
Gassy betty(18, true);
```

```
Gassy alan; // OK!!!
```

```
}
```

Constructor Basics

```
class Gassy
{
public:
    Gassy() // generated by compiler
    {
        // If (m_ageBeans == true)
        // return(10); So be careful!!!
    } return(3 * m_age);
}

private:
    int m_age;
    bool m_ageBeans;
};
```

These variables will
random values!

If you **don't** define any
constructors at all...

Then C++ generates an **implicit**,
default constructor for you.

Why would it do this? Well,
we'll see why in a bit. 😊

But this default constructor
won't initialize your object's
primitive member variables!

So be careful!

Primitive variables
include native C++
types like **int**, **float**,
double, **bool**, etc.

Non-primitives include
string, **Gassy**, etc.

erHr()); **//??**

When Constructors

```
int main()  
{
```

```
Gassy carey(45,false), bill;
```

```
Gassy arr[52];
```

```
Gassy *ptr = new Gassy(1,false);
```

```
for (int i=0; i < 10; ++i) {
```

```
    Gassy temp;
```

```
    ...
```

```
}
```

```
Gassy *justAPtr;
```

```
}
```

A constructor is called any time you **create a new variable** of a class.

A constructor is called **N times** (once for each array element) when you create an **array of size N**.

A constructor is called when you use **new** to **dynamically allocate a new variable**.

If a variable is declared **in a loop**, it is newly constructed during **EVERY iteration!**

The constructor is **not called** when you just **define a pointer variable!**

Destructors

Just as every class has a **constructor**, every class also has a **destructor function** (it may have *only one* of these).

The job of the **destructor** is to de-initialize or deconstruct a class variable when it **goes away**.

Here's how we define a **destructor** for our class...

```
class Gassy
{
public:
    ~Gassy()
    {
        for your destructor
        // code goes here
    }
private:
    ...
};
```

Destructors must **NOT** have
any **parameters**.
name of the class.

It looks just like a
constructor function

Destructors must **NOT**
return a **value** either.

Destructors

```
class Gassy
{
public:
    Gassy()
    {
        m_age = 0;
        m_ateBeans = false;
    }

    int getNerdScore()
    {
        if(m_ateBeans == true)
            return(100);
        return(3 * m_age);
    }

private: // generated by compiler
    {int m_age;
    }
    bool m_ateBeans;
};
```

If you don't define your own destructor for a class...

Then C++ will define an implicit one for you...

This ensures that objects of your class properly go away when they go out of scope.

Why Do We Need Destructors?

```
class SpotifyPlayer
{
public:
    SpotifyPlayer()
    {
        // Reserve 100MB of disk space to temporarily
        // cache downloaded MP3 music files.
        reserveSpaceOnDisk(100000000);
    }
};
```

We reserve 100MB of disk space when we construct a new Spotify object.

But nothing in our class ever frees this disk space!

So every time we create a new object, we waste **another** 100MB!

no reserved space on disk.

```
int main()
{
    // Obsessively play k-pop music!
    for (int i = 0; i < 100; ++i)
    {
        SpotifyPlayer a;
        a.playSong("http://spotify.com/dad.mp3");
    }
}
```

+100MB

Why Do We Need Destructors?

```
class SpotifyPlayer
```

```
{  
public:
```

```
    SpotifyPlayer()  
    {
```

```
        // Reserve 100MB of disk space to temporarily
```

```
        // cache downloaded MP3 music files.
```

```
        reserveSpaceOnDisk(100000000);  
    }
```

```
→ ~SpotifyPlayer()  
void playSong(string songURL)  
{
```

```
    // Free the reserved space on disk.
```

```
→ // First download song to reserved space on  
    freeReservedSpace();  
    downloadMP3ToReservedSpace();  
    // Then play the song that was downloaded  
    playMP3InReservedSpace();  
}
```

```
};
```

Now every time we construct we reserve 100MB...

And every time we destruct we free our 100MB!

course!

```
int main()
```

```
{  
    // Obsessively play k-pop music!
```

```
→ for (int i = 0; i < 100; ++i)
```

```
    {  
        +100MB
```

```
→ SpotifyPlayer a;
```

```
→ a.playSong("http://spotify.com/dad.mp3");
```

```
→ }
```

```
}
```

When must you have a destructor?

Any time a class **allocates**
a system resource...

Reserves memory using
the *new* command

Opens a disk file

Connects to another computer
over the network

Your class **must have a**
destructor that...

Frees the allocated memory
with the *delete* command

Closes the disk file

Disconnects from the
other computer

Don't forget or you'll **FAIL!**



Destructor

So when is a destructor called?

Local objects defined in a function are destructed when they "go out of scope."

Local variables defined in a block are destructed when the block finishes.

```
void Dingleberry()  
{
```

```
    SomeClass a;
```

```
    for (int j=0;j<10;j++)  
    {
```

```
        SomeClass b;  
        // do something
```

```
    } ← b's destructor is called each time we hit the end of the block
```

```
    MP3Player *m = new MP3Player;  
    m->getSong("www.music.com/x.wav");  
    m->playSong();
```

```
    delete m; ← m's destructor is called here  
                (if you don't use delete, m's d'tor is never  
                called - not even when the program ends!)
```

```
    } ← a's destructor is called here
```

Dynamically allocated variables (e.g., allocated using the **new** command) are destructed when **delete** is called, **before** the memory is actually freed by the OS.

Class Composition

What's the big picture?

Class composition is when a class contains one or more member variables that are also classes.

```
class Player {  
    ...  
};  
  
class Enemy {  
    ...  
};  
  
class VideoGame {  
    ...  
private:  
    Player user_hero;  
    Enemy enemies[10];  
};
```

The important thing to watch out for in the next set of slides is the process/syntax C++ uses to initialize these member variables.

Class
Composition

Why
should
I care?



Uses:

Class composition is required in most C++ classes - you build classes using other classes!

```
class Stomach {
public:
    Stomach() { myGas = 0; }
    void eat() { myGas ++; }
}; ...
```

```
class Brain {
public:
    Brain() { myIQ = 100; }
    void think() { myIQ += 10; }
}; ...
```

```
class HungryNerd {
public:
    HungryNerd()
    {
        myBelly.eat();
        myBrain.think();
    }
private:
    Stomach myBelly;
    Brain   myBrain;
};
```

Then C++ runs the outer class's constructor!

C++ constructs members first, from top to bottom.

Class Composition

So how does **construction** work when we use class composition?

Let's look at three classes...

COMPOSITION CONSTRUCTION RULE:

C++ always constructs member variables first.

Then C++ constructs the overall/outer object last.



```
class Stomach {
```

```
public:
```

```
→ Stomach() {
```

```
void eat() { ... }
```

```
}; ...
```

```
class Brain {
```

```
public:
```

```
→ Brain() {
```

```
void think() { myIQ = 10; }
```

```
};
```

```
class HungryNerd {
```

```
public:
```

```
→ HungryNerd() {
```

```
→ Call 's default constructor  
→ Call 's default constructor
```

```
→ myBrain.think();
```

```
→ }
```

```
private:
```

```
→ Stomach myBelly;
```

```
→ Brain myBrain;
```

```
};
```

```
myBelly
```

```
myBrain
```

Default
constructor
(Has no parameters)

Default
constructor
(Has no parameters)

This code is automatically
added by the compiler!

Member
objects

How does this actually work?

When an outer object contains
member objects...

C++ secretly adds code to the outer
class's constructor to FIRST call the
DEFAULT constructors of all
the member objects...

And then AFTER they're
created, C++ runs the body of
the object's constructor.

carey

myBelly myGas 0

myBrain myIQ 100

```
int main()
```

```
{
```

```
→ HungryNerd carey;
```

```
→ ...
```

```
}
```

```
class Stomach {
public:
    Stomach() { myGas = 0; }
    void eat() { myGas ++; }
}; ~Stomach() { cout << "Fart!\n"; }
```

```
class Brain {
public:
    Brain() { myIQ = 100; }
    void think() { myIQ += 10; }
}; ~Brain() { cout << "Argh!\n"; }
```

```
class HungryNerd {
public:
    → ~HungryNerd()
    {
        myBelly.eat(); // last meal
        myBrain.think(); // last thought
    }
private:
    Stomach myBelly;
    Brain myBrain;
};
```

OK, what about **destruction**?

At the time when our object (carey) goes out of scope, C++ automatically calls its destructor.

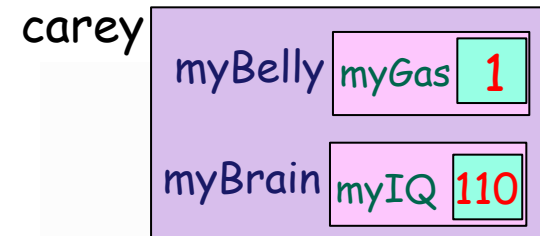
DESTRUCTION RULE:

C++ always runs the outer class's destructor first.

Then C++ runs the destructors of the member variables (in reverse order).

C++ runs the outer destructor first!

Then C++ destructs member variables (in reverse order)!



C++ calls carey's destructor

Multi-level Composition Construction/Destruction

```
class Food
{
public:
    Food()
    ~Food()
};
```



Here's the order the c'tors will run for three levels of classes...

And here's the order the destructors will run...

```
class Stomach
{
public:
    Stomach()
    ~Stomach()
private:
    Food    myFood;
};
```



```
class Nerd
{
public:
    Nerd()
    ~Nerd()
private:
    Stomach    myStomach;
};
```



david_s

myStomach

myFood

"Turkey"

```
int main()
{
    → Nerd    david_s;
    ...
    → }
```

Advanced Class Composition

Of course, things are never quite so simple...

Let's look at a slightly more complex
class composition example.

Let's change the **constructor** of our Stomach class so it
REQUIRES the user to specify the **amount of gas** each
stomach starts with.

Advanced C++

Ok, so now our new `Stomach` constructor **REQUIRES** us to pass in a value

And here's how we create a new `Stomach` variable...

```
class Stomach
{
public:
    Stomach(int myGas) {}
    ~Stomach() { cout << "Fart!\n"; }
    void eat() { myGas++; }
private:
    int myGas;
};
```

```
int main()
{
    Stomach a(5); // 5 farts
    Stomach b; // ???

    a.eat(); // 6 farts
    ...
}
```

Now, what happens if we want to use our new `Stomach` class in our `HungryNerd` class?

Advanced Class Composition

```
class Stomach
{
public:
    Stomach(int startGas)
    {
        myGas = startGas;
    }
    ~Stomach() { cout << "Fart!\n"; }
    void eat() { myGas++; }
private:
    int myGas;
};
```

```
class HungryNerd
{
public:
    HungryNerd()
    {
        myBelly.eat();
        myBrain.think();
    }

    ...

private:
    Stomach myBelly;
    Brain myBrain;
};
```

Will our HungryNerd class still compile?

NO!

Advanced Composition

Because our new **Stomach** class **REQUIRES** you to **pass a value** in to its **constructor**.

```
class Stomach
{
public:
    Stomach(int startGas)
    {
        myGas = startGas;
    }
    ~Stomach() { cout << "Fart!\n"; }
    void eat() { myGas++; }
private:
    int myGas;
};
```

But our Stomach class doesn't have a default (parameter-less) constructor anymore!

```
public:
    HungryNerd()
    {
        Call myBelly's default constructor
        Call myBrain's default constructor
        myBrain.think();
    }
```

But our auto-generated C++ code **always** calls the **default constructor**, which takes **NO parameters**!

So this results in a C++ **compilation error**!

Advanced

And pass in a value so it can be properly constructed!

```
class Stomach
{
public:
    Stomach(int startGas)
    {
        myGas = startGas;
    }
    ~Stomach() { cout << "Fart!\n"; }
    void eat() { myGas++; }
private:
    int myGas;
};
```

po

Well, our HungryNerd constructor needs to somehow explicitly call myBelly's constructor...

```
class HungryNerd
{
public:
    HungryNerd()
    {
        myBelly.eat();
        myBrain.think();
    }
private:
    Stomach myBelly;
    Brain myBrain;
};
```

Call myBelly's default constructor
Call myBrain's default constructor

Ok, so how can we **fix** our HungryNerd class so it works with our new Stomach class?

Composition

whose constructor requires parameters!

We no longer going to let call myBelly's default constructor for us...

```
class Stomach
{
public:
    Stomach(int startGas)
    {
        myGas = startGas;
    }
    ~Stomach() { cout << "Fart!\n"; }
    void eat() { myGas++; }
private:
    int myGas;
};
```

Instead, we'll add our own C++ code to explicitly call myBelly's constructor with a parameter.

```
class HungryNerd
```

```
{
public:
    HungryNerd()
    {
        : myBelly(10)
        Call myBelly's default constructor
        Call myBrain's default constructor
        myBelly.eat();
        myBrain.think();
    }
private:
    Stomach myBelly;
    Brain myBrain;
};
```

This is called an "initializer list".

Call myBelly's default constructor
Call myBrain's default constructor

We use it to initialize an embedded object...

Alright, let's see the "exciting" C++ syntax!

Advanced Class Composition

```
class  
{  
public:  
    St  
    {  
        myGas = startGas;  
    }  
    ~Stomach() { cout << "Fart!\n"; }  
    void eat() { myGas++; }  
private:  
    int myGas;  
};
```

The **initializer list** sits between the constructor's prototype and its body.

myGas = startGas;

```
class HungryNerd  
{  
public:  
    HungryNerd()  
        : myBelly(10)  
    {  
        myBrain.eat();  
    }  
private:  
    Stomach myBrain;  
};
```

You **must** add an **initializer list** to **all** of your outer class's constructor(s).

Call myBrain's default constructor

It starts with a **colon**, followed by one or more **member variables** and their **parameters** in parentheses.

Any time you have a member variable (e.g., **myBelly**) that **requires a parameter** for construction...

So here's what it looks like (without the C++ magic).

Advanced Class Compos

```
class Stomach
```

```
{  
  class Brain  
  {  
  public:  
    → Brain() { myIQ = 100; }  
    void think() { myIQ += 10; }  
  };  
};
```

Finally, once all of our embedded objects have been constructed, C++ will run the outer constructor's body!

```
class HungryNerd
```

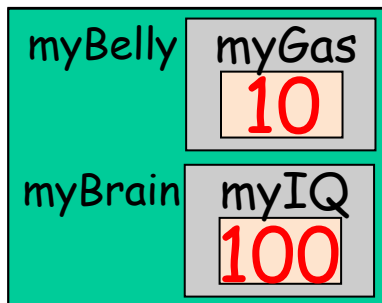
```
{  
  public:  
    → HungryNerd() {  
      → myBelly(10)  
      → Call myBrain's default constructor  
    }  
    myBelly.eat();  
    myBrain.think();  
  }  
  private:  
    Stomach myStomach;  
    Brain myBrain;  
};
```

This line calls *myBelly's* constructor with a parameter value of **10**!

Of course, C++ is still happy to implicitly construct any embedded objects with default constructors for us!

see our new
HungryNerd
and *Stomach*
classes in
action!

carey



```
int main()  
{  
  → HungryNerd  carey;  
  ...  
}
```

Advanced C++

Our new initializer list explicitly calls both **Stomach's** and **Brain's** constructors with parameters!

So C++'s implicit call to the default constructor **will no longer work!**

Let's fix it!

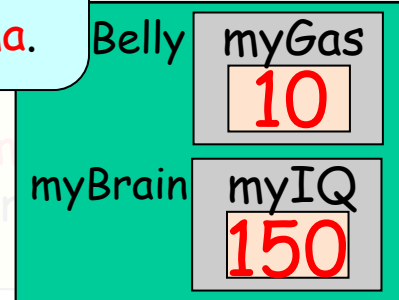
```
class Stomach
{
public:
    Brain() { myIQ = 100; }
    void think() { myIQ += 10; }
};
```

```
public:
    HungryNerd()
        : myBelly(10), myBrain(150)
    {
        // Call myBrain's default constructor
    }
    void think();
};
```

We just separate each member object and its parameters with a **comma**.

Finally, once all of the embedded objects have been constructed, C++ will run the outer constructor's body!

```
class HungryNerd
{
public:
    HungryNerd()
    {
        myStomach = Stomach();
        myBrain = Brain();
    }
    void think() { myIQ += 10; }
};
```



```
int main()
{
    HungryNerd carey;

    ...
}
```

Initializer Lists

If you like, you **may** also use the initializer list to **initialize** your **primitive member variables** too.

```
class HungryNerd
{
public:

    HungryNerd()
    : myBelly(10), myBrain(150),
    {
        myBelly.eat();
        myAge = 19;
    }

private:
    Belly myBelly;
    Brain myBrain;
    int myAge;
};
```

You may do it **here** in your initializer list...

()

Feel free to use the initializer list for primitives if you like,

This will initialize your primitive variable(s) before the body of your constructor runs.

Instead of initializing your primitive variables **here** in your **constructor body**...

Advanced Class Comp

We don't need to have constants here!

```
class Stomach
{
public:
→ Stomach(int startGas)
{
→ myGas = startGas;
→ }
```

```
class HungryNerd
{
public:
→ HungryNerd(int startingGas)
→ myBelly(startingGas*4)
→ Call myBrain's default constructor
```

75

Now we'll initialize our myBelly member with whatever the user passes in to the HungryNerd constructor!

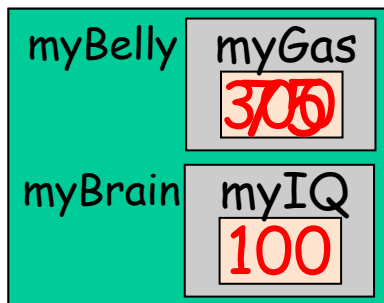
And these can even be more interesting expressions!

```
~Stomach() { cout << "Stomach destroyed" << endl; }
void eat() { myGas--; }
private:
int myGas;
};
```

```
Stomach myBelly;
Brain myBrain;
};
```

Finally, there's no reason why the parameters to your initializer list have to be constants...

carey



```
int main()
{
→ HungryNerd carey(75);
...
}
```

A Few Topics

Let's talk about four topics that you'll need in order to solve your projects...

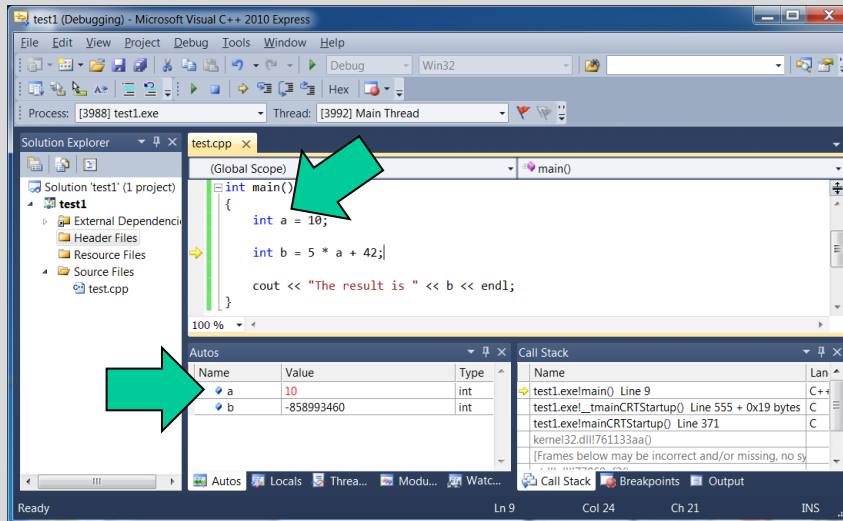


Learning to Use the C++ Debugger

What's the big picture?

Visual Studio/Xcode debuggers help you find bugs 10x faster!

You can trace programs line-by-line and see the value of every variable (like we do in slides)!



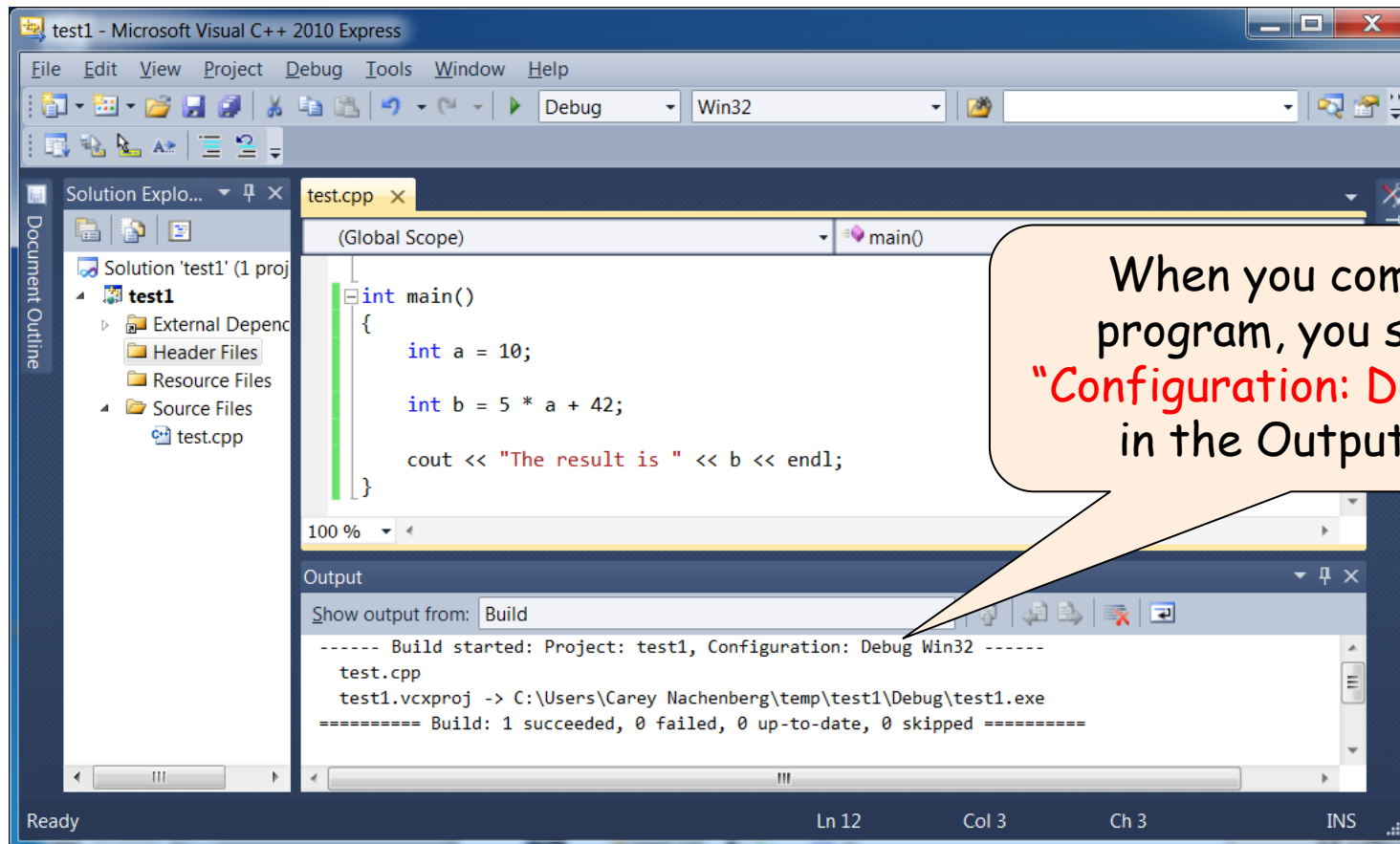
Uses:

Use the debugger to quickly figure out why your code is crashing, hanging or giving the wrong result.

It takes about 10 mins to learn how the Visual Studio/Xcode debuggers work, so get going!

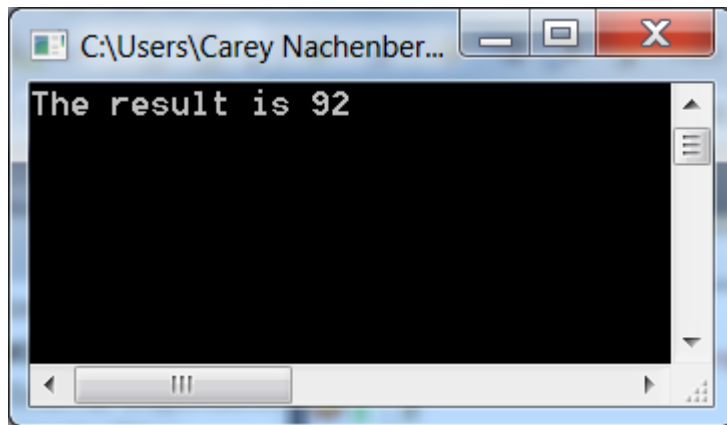
Debugging

By default, Visual Studio compiles all programs in "debug mode" so you can use the debugger with them right away.

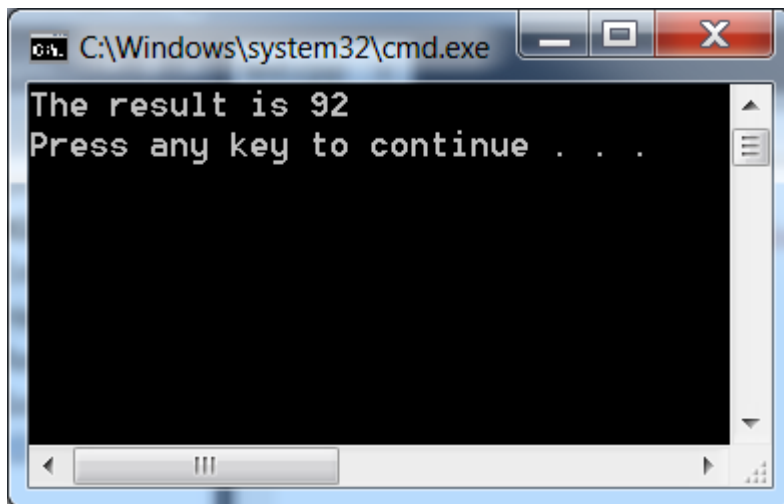


Debugging

You can run your program from start to finish by hitting either **F5** or **Ctrl-F5**.



If you run your program by hitting **F5**, Visual Studio will immediately close the debug window after your program finishes.



If you run by hitting **Ctrl-F5**, Visual Studio will print **"Press any key to continue..."** and leave the debug window on the screen when your program completes.

Debugging

You can step through your program line by line by hitting the **F10** and **F11** keys.

To start your program and debug one line at a time, hit **F10** after you finish compiling it.

To run the current line of code, hit **F10** or **F11**.

Let's do that now.

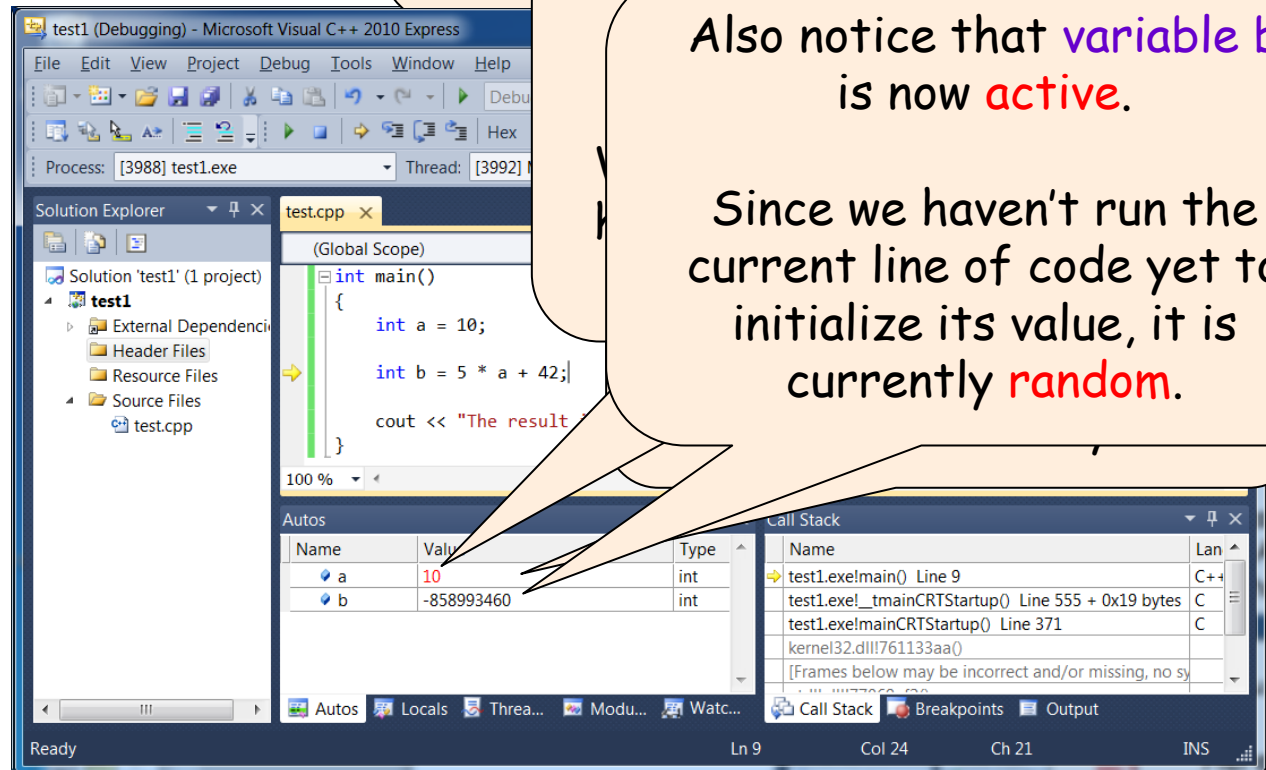
Let's hit **F10** again and trace over another line.

You'll see this little yellow arrow next to the first line of your program.

The arrow points to the line of code that's ABOUT to be executed.

Also notice that **variable b** is now **active**.

Since we haven't run the current line of code yet to initialize its value, it is currently **random**.

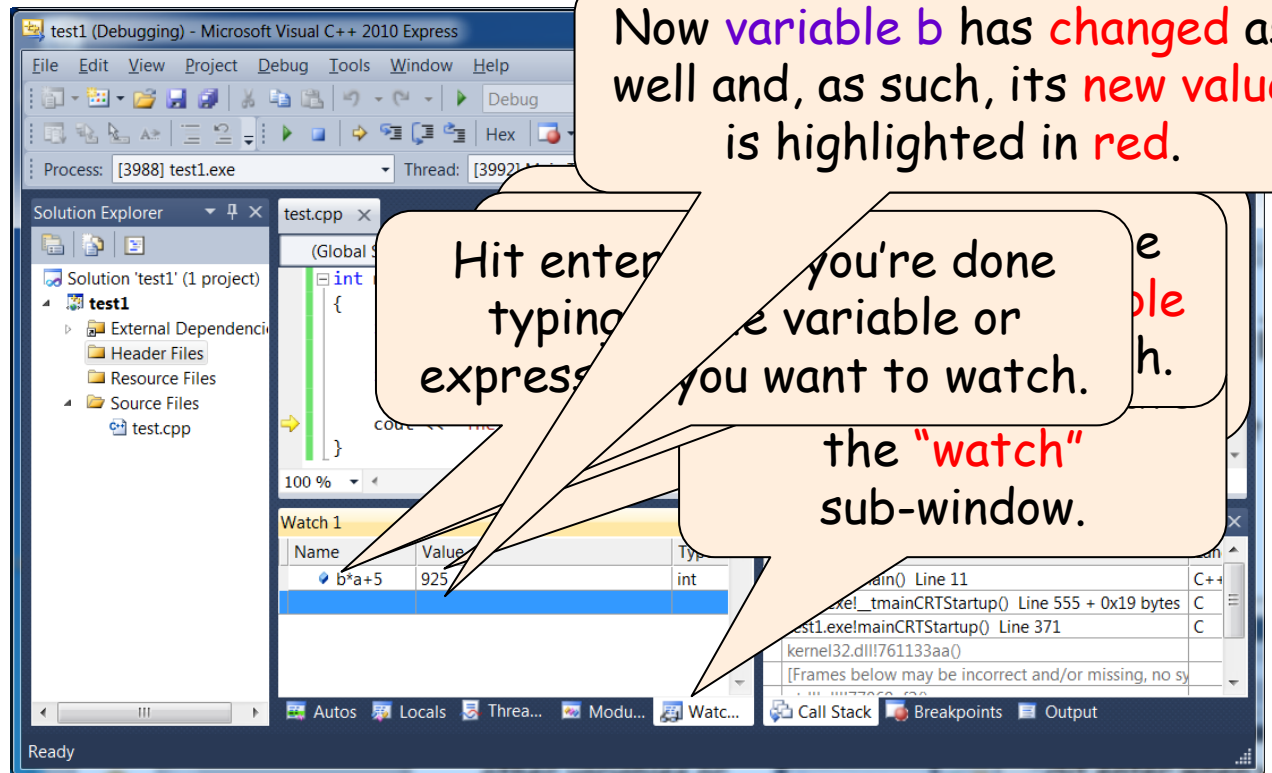


Debugging

You can step through your program one line at a time using the **F10** and **F11** keys.

Let's hit **F10** again and trace over another line.

If you want to watch other variables or expressions, you can do that in the **"watch"** sub-window.



Debug

The **F10** and **F11** keys actually do slightly different things...

If you hit **F10** while on a function call line, Visual Studio will run the entire function in one step. You won't be able to trace line-by-line through it.

So **F10** lets you quickly run a line of code without delving into the details.

The screenshot shows the Visual Studio IDE in a debug state. The main window displays the source code of `test.cpp` with the following content:

```
foo();  
cout << "a has a value of: " << a << endl;  
}
```

The cursor is positioned on the line `cout << "a has a value of: " << a << endl;`. The `Output` window at the top right shows the text `c has a value of: 10`. The `Solution Explorer` on the left shows the project structure for `test1`. The `Autos` window at the bottom left shows the variable `a` with a value of `10`. The `Call Stack` window at the bottom right shows the current call stack, with `test1.exe!main() Line 16` at the top.

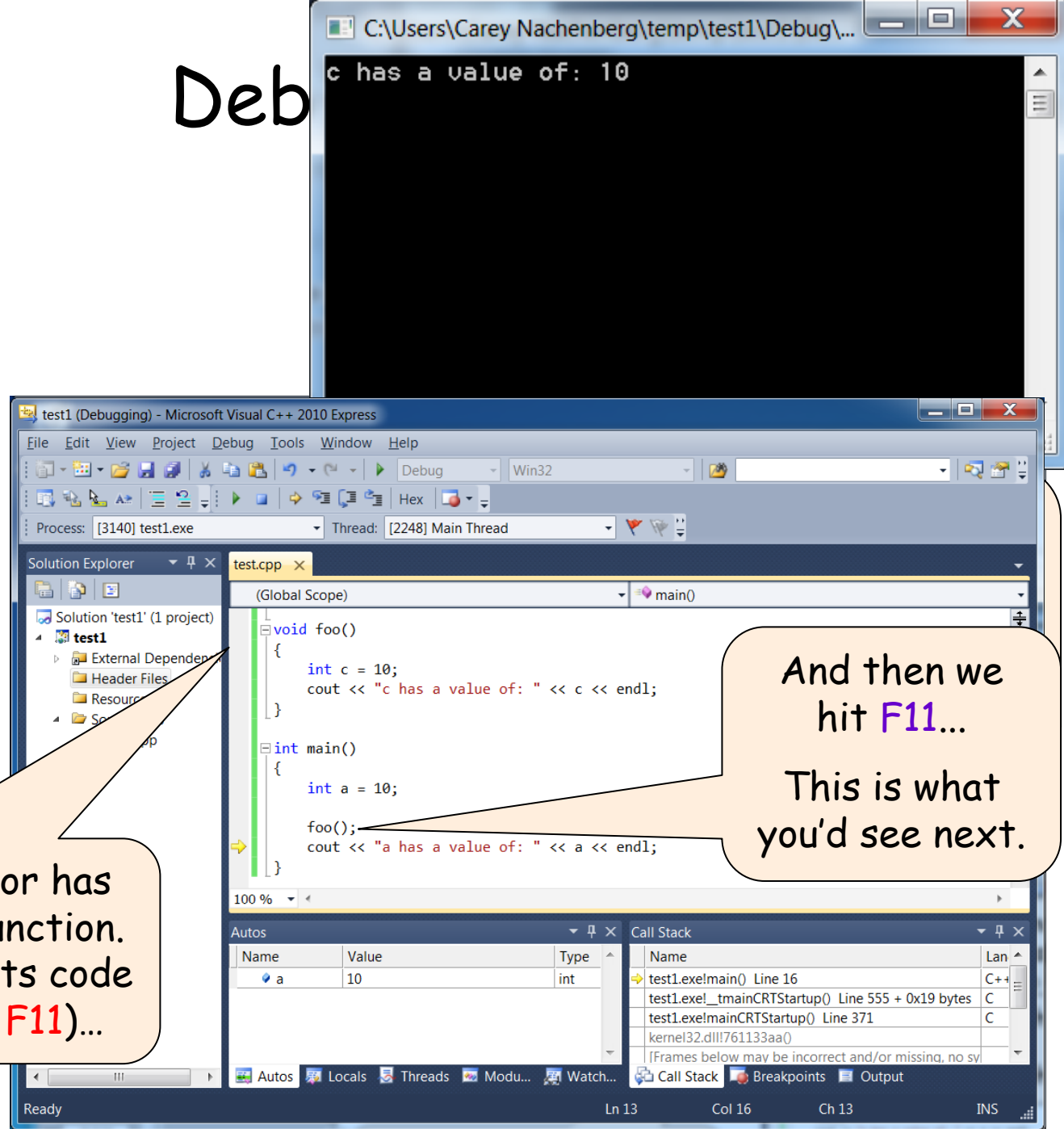
Callouts provide additional context:

- A callout from the `Output` window states: "Notice that Visual Studio ran the entire **foo()** function from start to finish..."
- A callout from the code editor states: "Let's imagine we've been tracing a program and are about to execute this line:"
- A callout from the code editor states: "And now our cursor is on the line after the function call. then we **F10**..."
- A callout from the code editor states: "This is what you'd see next."

Deb

If you hit **F11** while on a function call line, Visual Studio will let you step into that function and trace through it a line at a time.

So **F11** lets you dive into a function call and trace through it. Notice our debug cursor has moved into the `foo()` function. Now we trace through its code line by line as well too (by hitting **F10** or **F11**)...



Deb

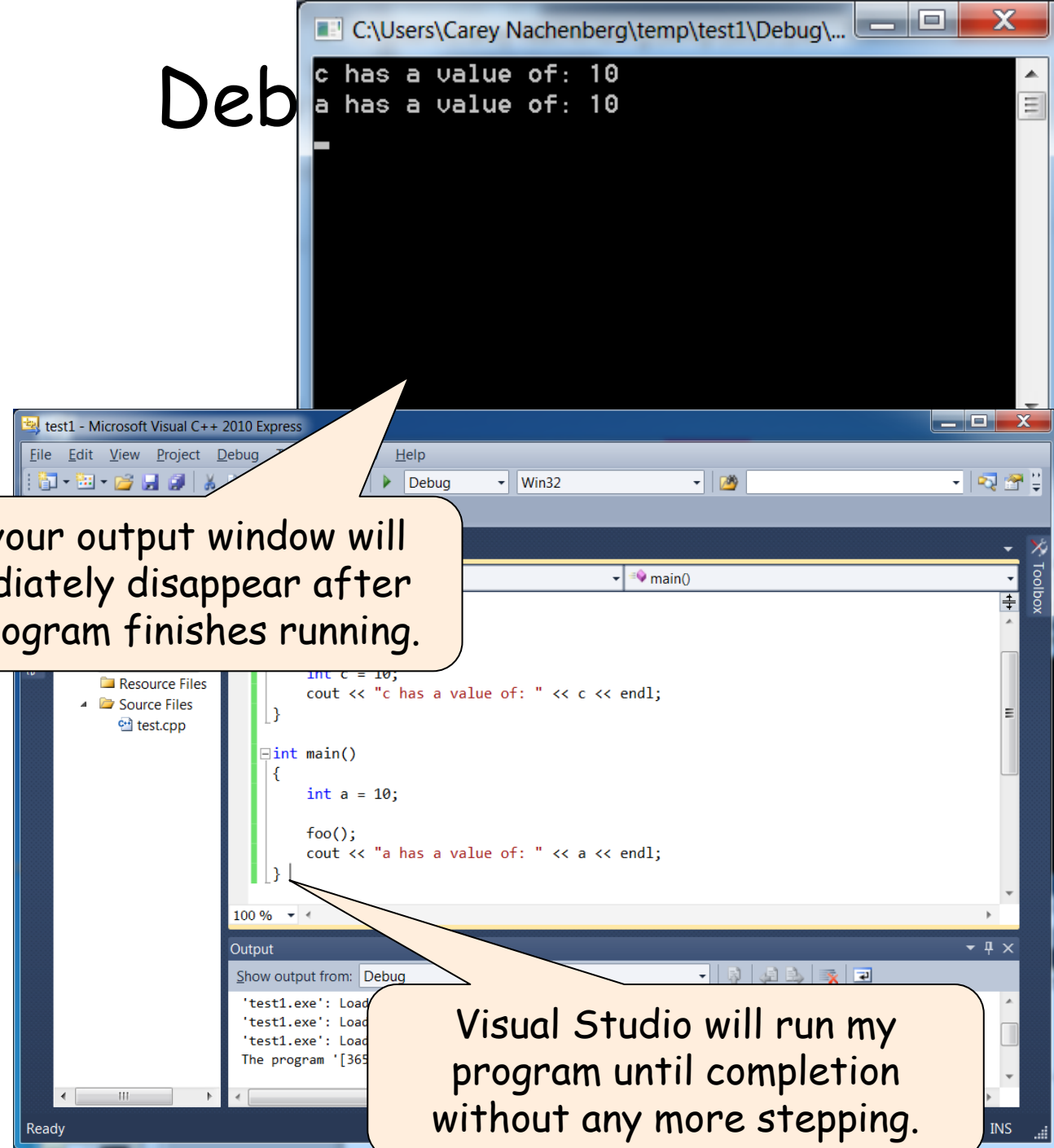
If at any time while
you're tracing
line-by-line you want
to stop tracing
line-by-line...

And simply let
rest of your pr
run normally...

Just hit **F5**
and Visual Studio will
run your program
until completion!

And your output window will
immediately disappear after
the program finishes running.

Visual Studio will run my
program until completion
without any more stepping.



Your program will run until it hits the breakpoint.

Then the debugger stops the program and lets you step through it line-by-line...

Or if you like, hit **F5** again and your program will run until it hits the breakpoint again.

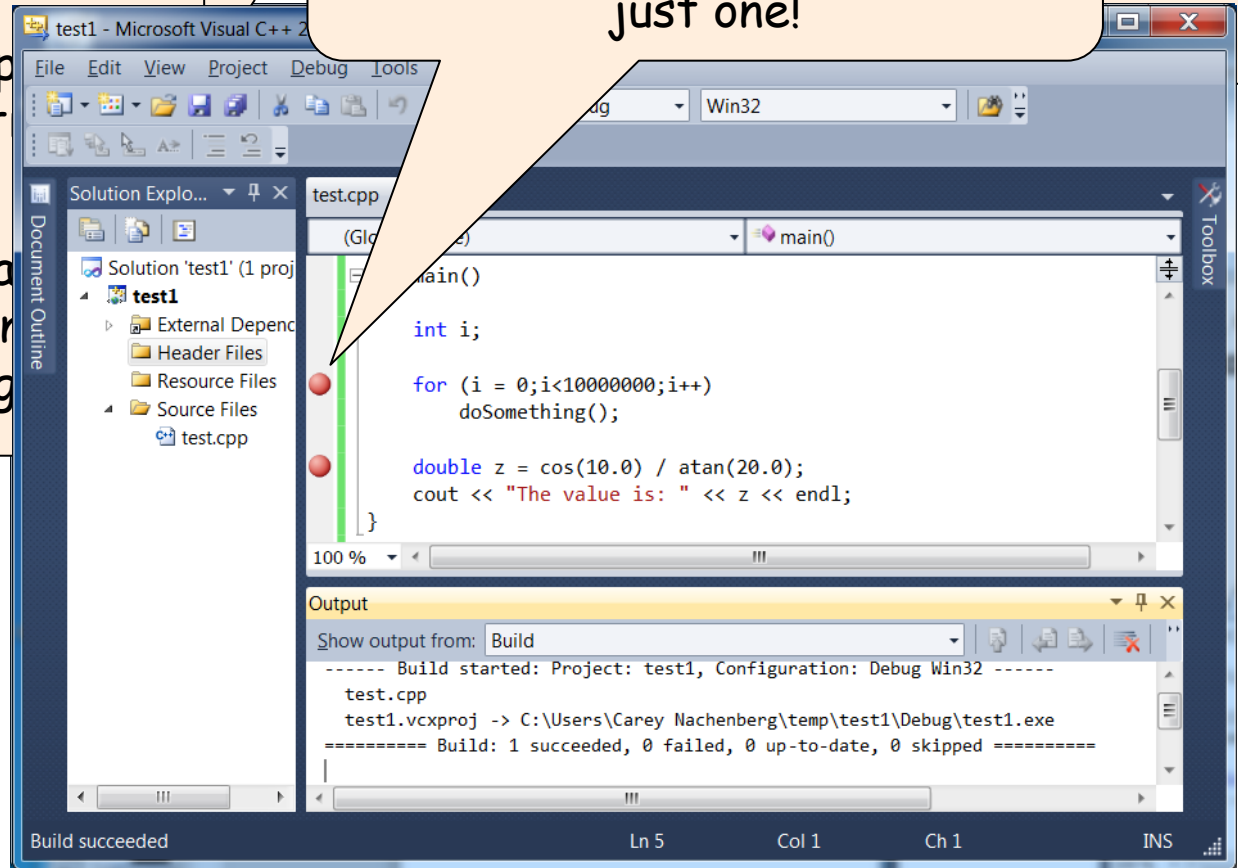
...or you can set a breakpoint at a likely location of a bug.

What should you do?

Answer:

Create a **breakpoint**!

Oh, and you can add as many breakpoints as you like... Not just one!



Go to the line where you'd like to start debugging and hit the **F9** key.

Then just hit the **F5** key (not Ctrl-F5) to run your program until it reaches that line!

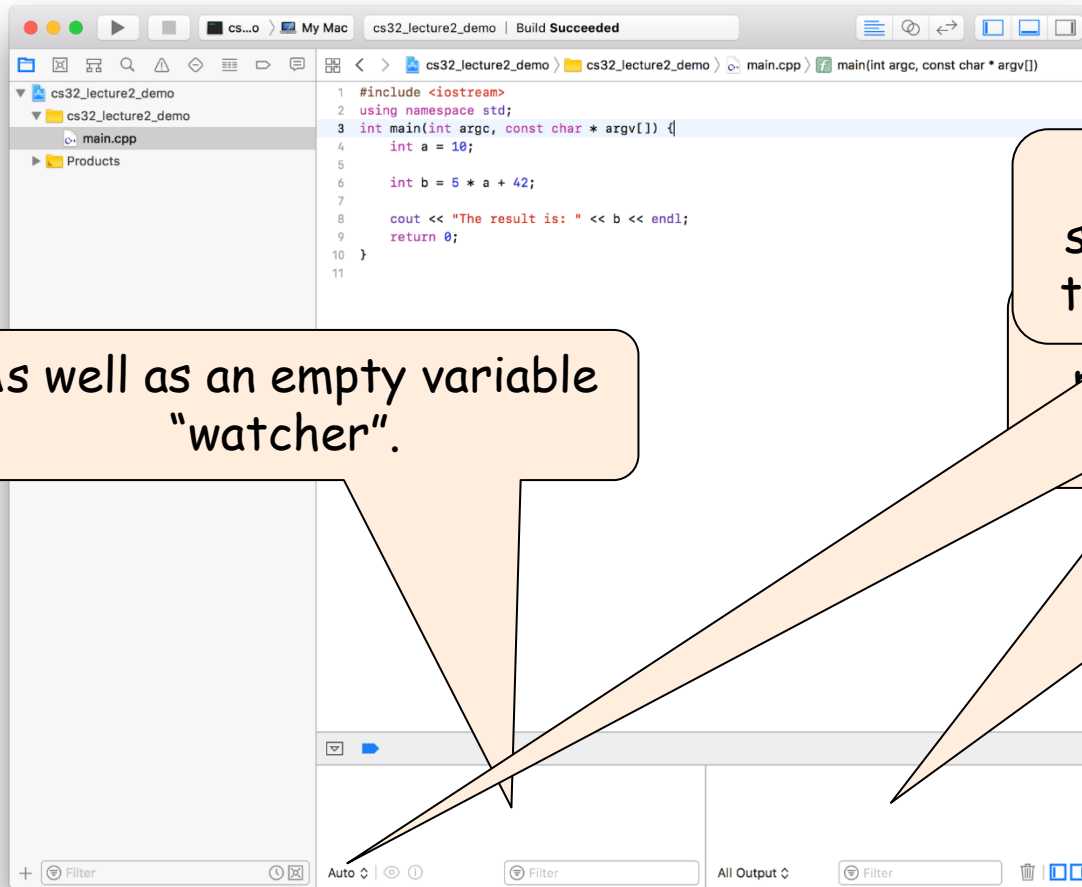
Debugging with Xcode

The **SUPERIOR** IDE ☺



Debugging with Xcode

This is Xcode's code editor. Xcode is an **IDE** created by **Apple**, specifically for iOS, macOS, tvOS, and watchOS development, but it also comes fully loaded with **C and C++ support**.



As well as an empty variable "watcher".

The "Auto" means that the sub-window will automatically track and show new variables.

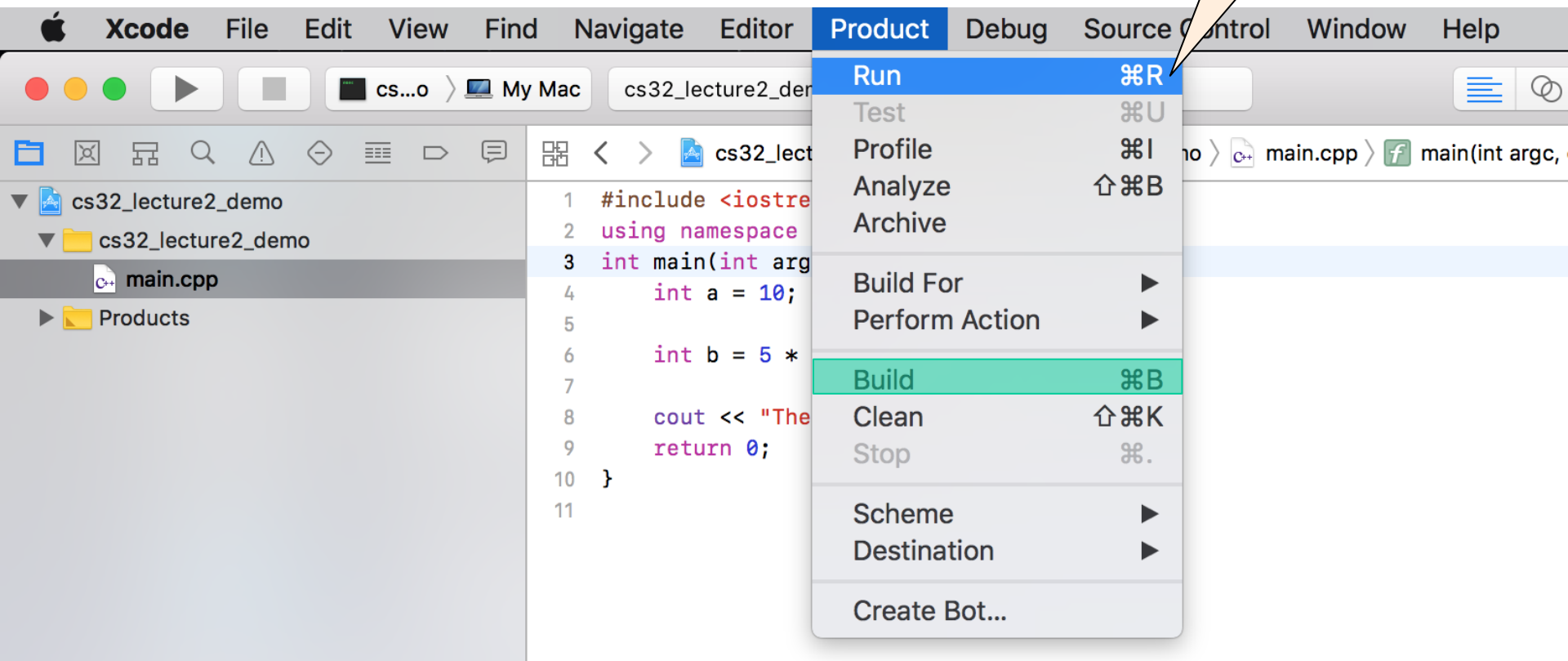
When you start a debug session, you'll see an empty debug output window...

Debugging with Xcode

You can run your program from start to finish by going to:

Product > Run

Xcode also has shortcuts 😊



Debugging with Xcode

You can also step through your program, one line at a time.

Let's insert a breakpoint at line 3.

```
1  #include <iostream>
2  using namespace std;
3  int main(int argc, const char * argv[]) {
4      int a = 10;
5
6      int b = 5 * a + 42;
7
8      cout << "The result is: " << b << endl;
9      return 0;
10 }
11
```

Just tap on the gray space next to the 3.

Debugging with Xcode

You can also step through your program, one line at a time.

Let's insert a breakpoint at line 3.

The blue arrow on line 3 shows that our breakpoint is **active**.

If we click on the breakpoint again, we **deactivate** it, but Xcode remembers that we had a breakpoint there and will let us turn it back on.
Neat!

```
1  #include <iostream>
2  using namespace std;
3  int main(int argc, const char * argv[]) {
4      int a = 10;
5
6      int b = 5 * a + 42;
7
8      cout << "The result is: " << b << endl;
9      return 0;
10 }
```

```
1  #include <iostream>
2  using namespace std;
3  int main(int argc, const char * argv[]) {
4      int a = 10;
5
6      int b = 5 * a + 42;
7
8      cout << "The result is: " << b << endl;
9      return 0;
10 }
11
```

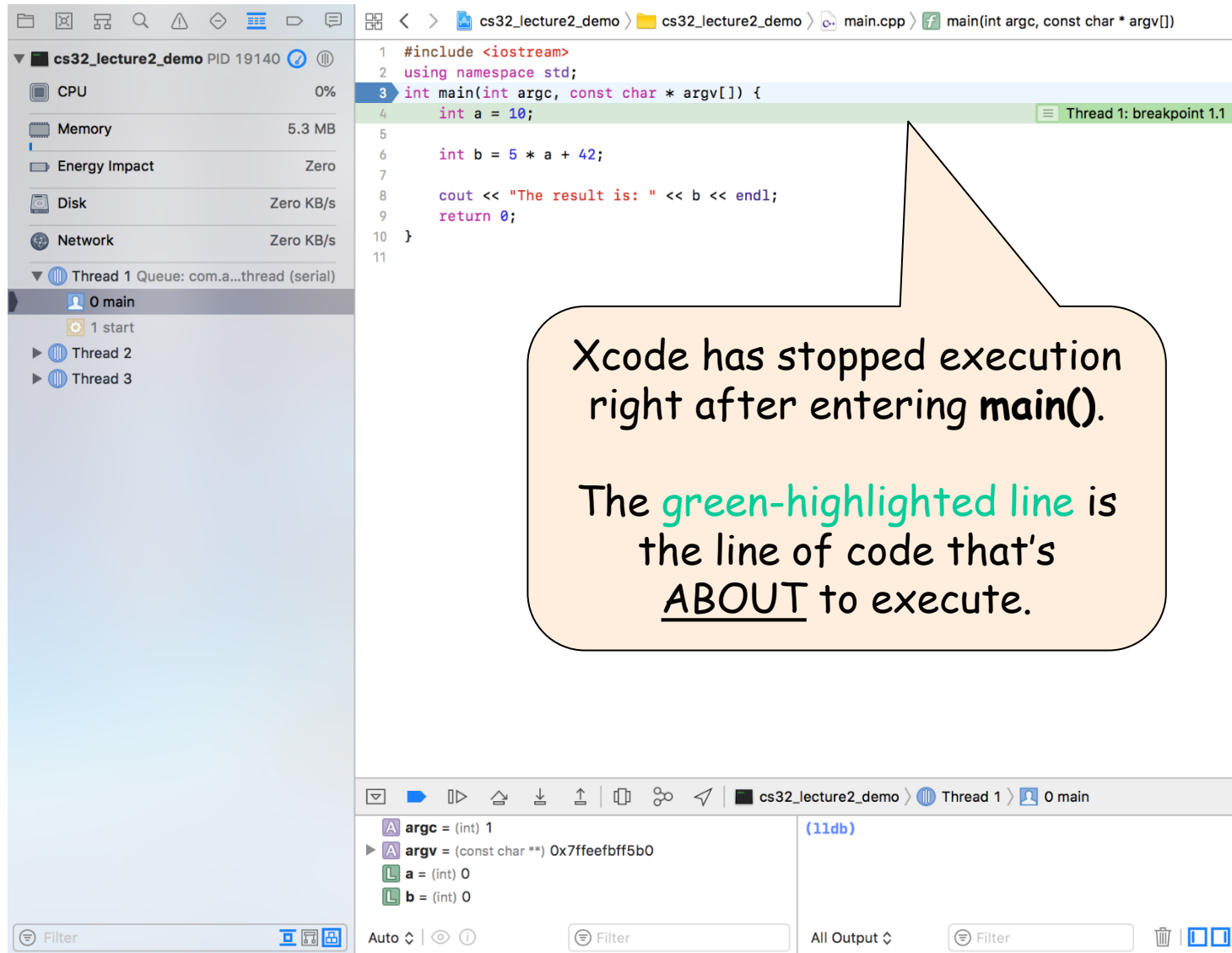
Debugging with Xcode

Because we set a breakpoint at `main()`, we can debug 1 line at a time.

This screenshot to the right shows what an Xcode debugging session looks like.

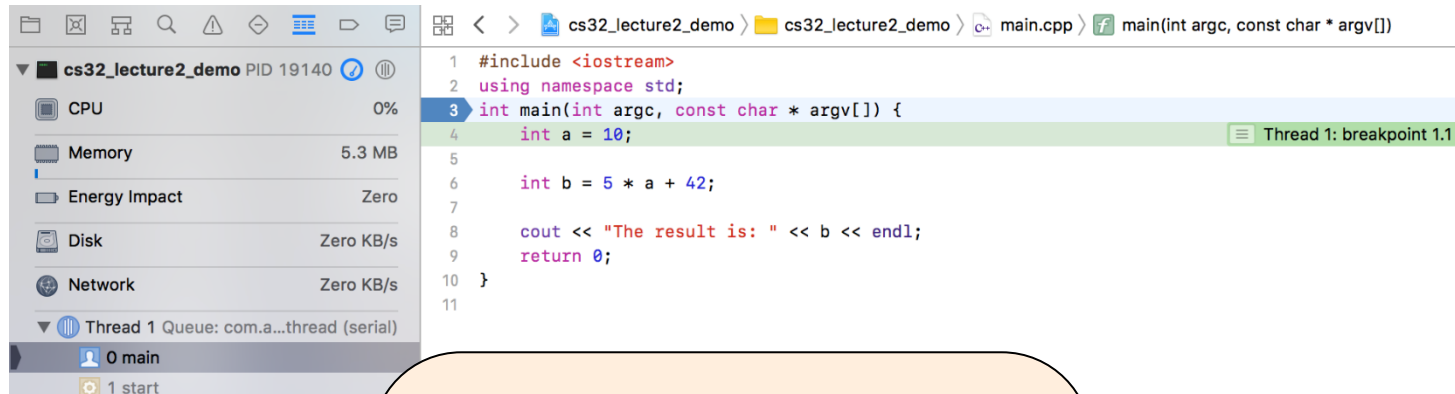
The left pane is showing resource usage.

The right pane is the code being run.



Debugging with Xcode

Because we set a breakpoint at `main()`, we can debug 1 line at a time.



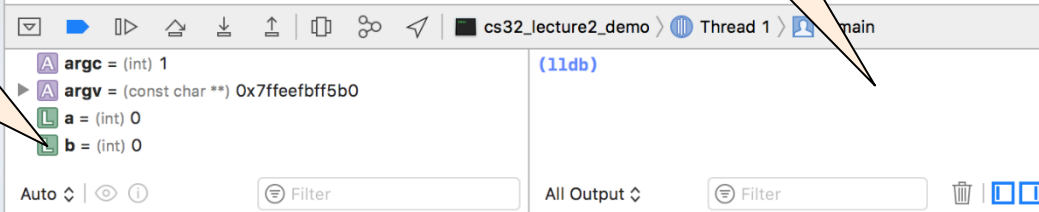
Our variable "watcher" has also indicated values for **a** and **b**.

The `a` and `b` variables appear to be initialized, but in C++ you can't depend upon that to be the case.

Always assume primitive values are uninitialized.

This window is our debug output. As we run `cout` statements, we'll see that output in this window.

Note: "lldb" is Xcode's command-line debugger. Ignore it for now.



Debugging with Xcode

Let's run the current line of code.

The screenshot shows the Xcode IDE interface. On the left, the 'Debug Console' pane displays system metrics for the process 'cs32_lecture2_demo' (PID 19140): CPU at 0%, Memory at 5.3 MB, Energy Impact at Zero, Disk at Zero KB/s, and Network at Zero KB/s. Below these, the 'Thread 1 Queue: com.a...thread (serial)' is expanded, showing '0 main' as the active thread, with '1 start' and 'Thread 2' and 'Thread 3' listed below it.

The main editor displays a C++ file 'main.cpp' with the following code:

```
1 #include <iostream>
2 using namespace std;
3 int main(int argc, const char * argv[]) {
4     int a = 10;
5
6     int b = 5 * a + 42;
7
8     cout << "The result is: " << b << endl;
9     return 0;
10 }
11
```

A breakpoint is set at line 4, indicated by a green bar and the text 'Thread 1: breakpoint 1.1'. The 'Step Over' button (a blue square with a right-pointing arrow) is highlighted in the debugger toolbar at the bottom.

A callout bubble points to the 'Step Over' button with the text: 'This is the "Step Over" button (more on that later). For now, just tap on that to run the current line of code.'

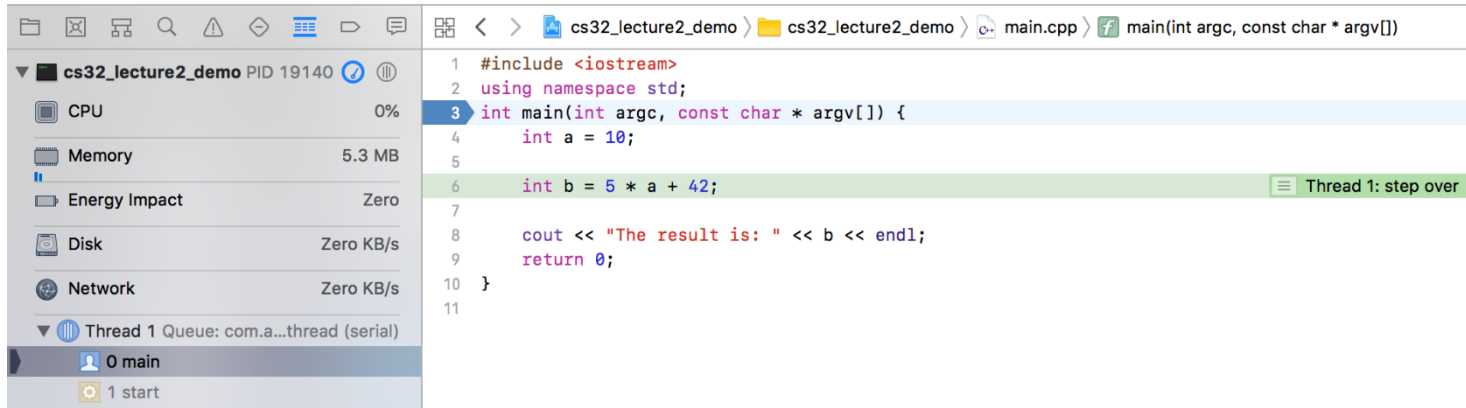
The bottom of the screen shows the 'Variables' pane with the following values:

- `argc` = (int) 1
- `argv` = (const char **) 0x7ffefbfff5b0
- `a` = (int) 0
- `b` = (int) 0

The 'Output' pane on the right is empty, showing '(lldb)' at the top.

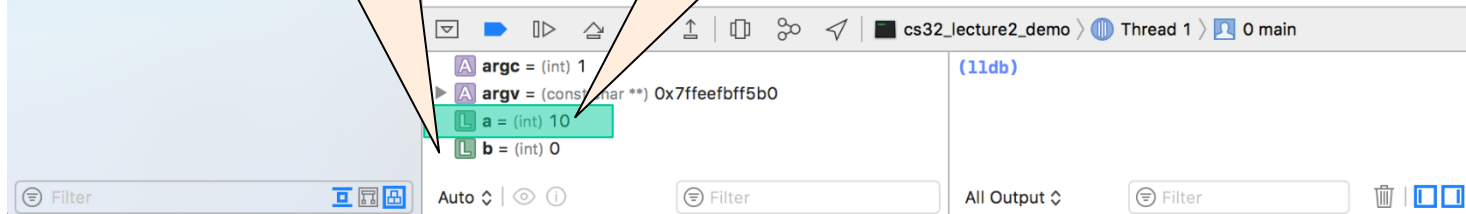
Debugging with Xcode

Let's run the current line of code.



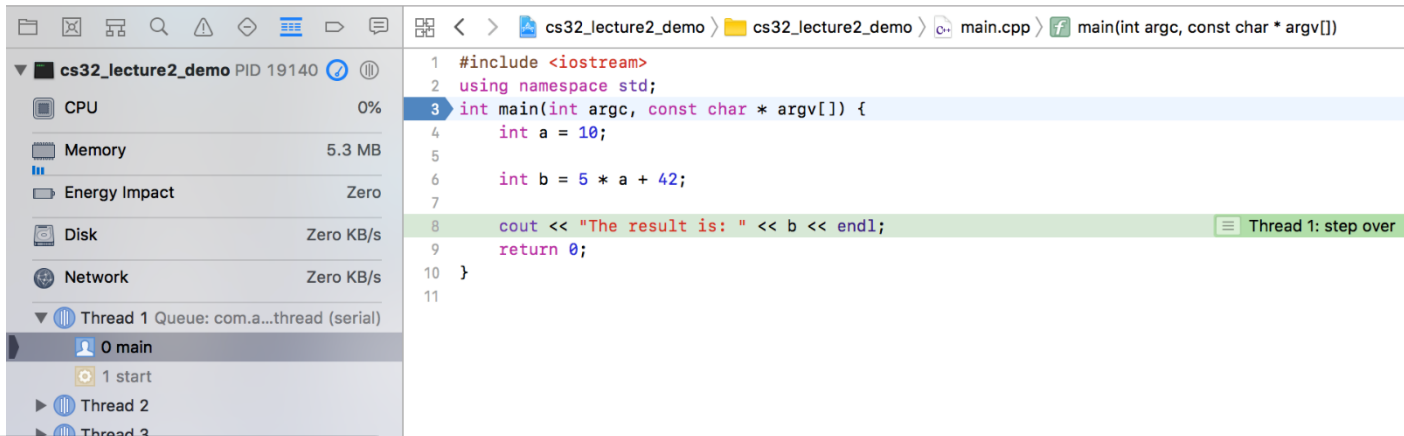
But **b** hasn't changed yet.
Let's press the **"Step Over"**
button again.

Check this out! The value for
a has changed because we
just executed the line of
code to set **a's** value.



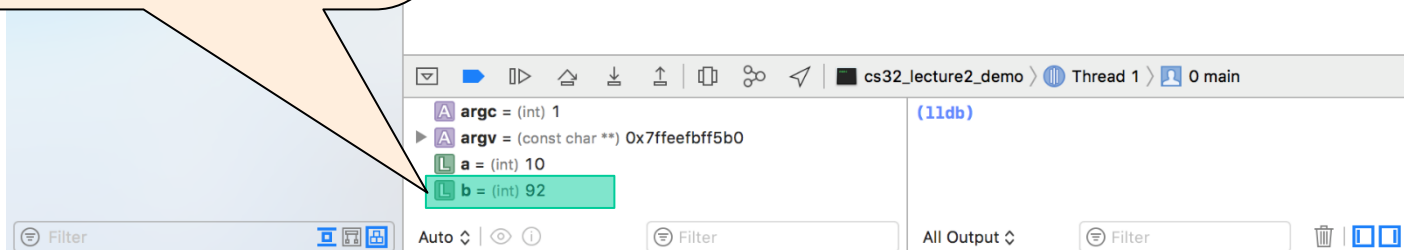
Debugging with Xcode

Let's run the current line of code.



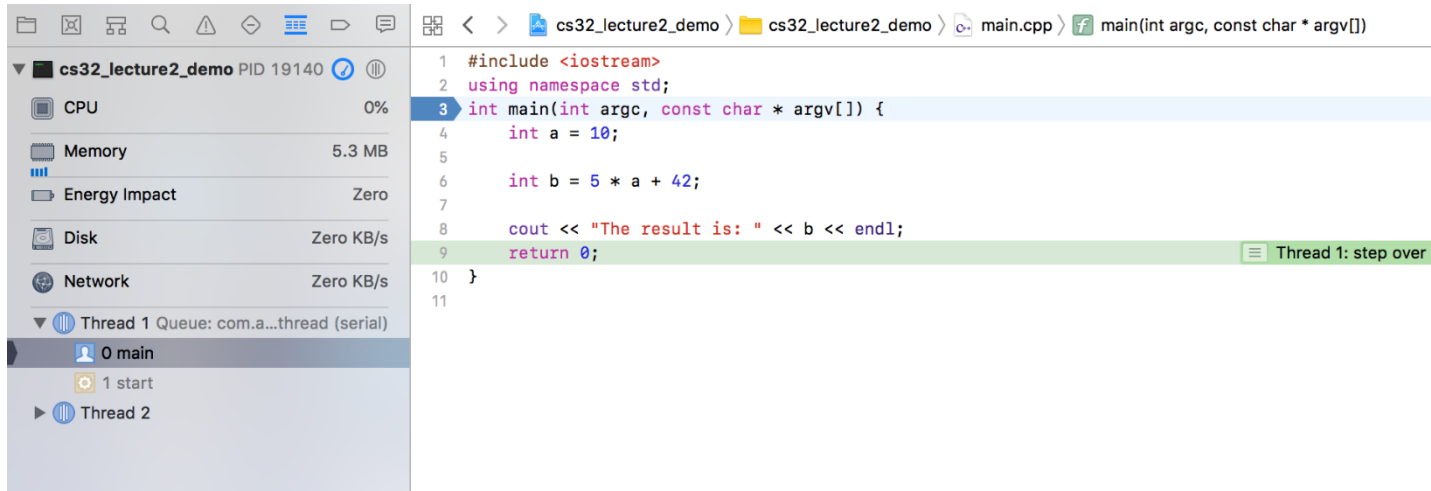
Now **b** has been updated,
just as we expected!

Tap on the **"Step Over"**
button again to see the **cout**
output.



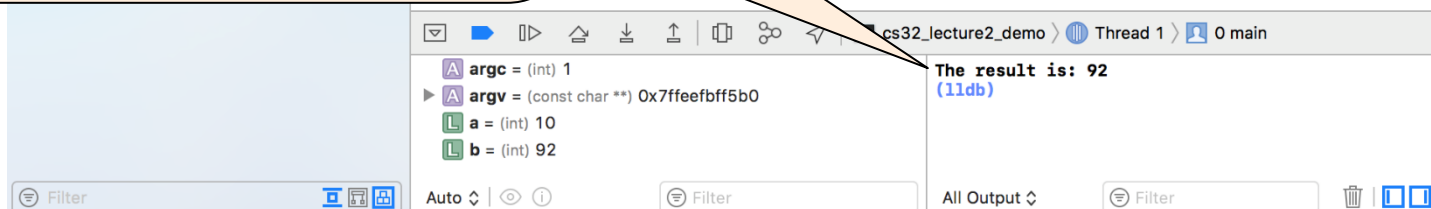
Debugging with Xcode

Let's run the current line of code.



Notice that the debug output window now has the **cout** statement's result.

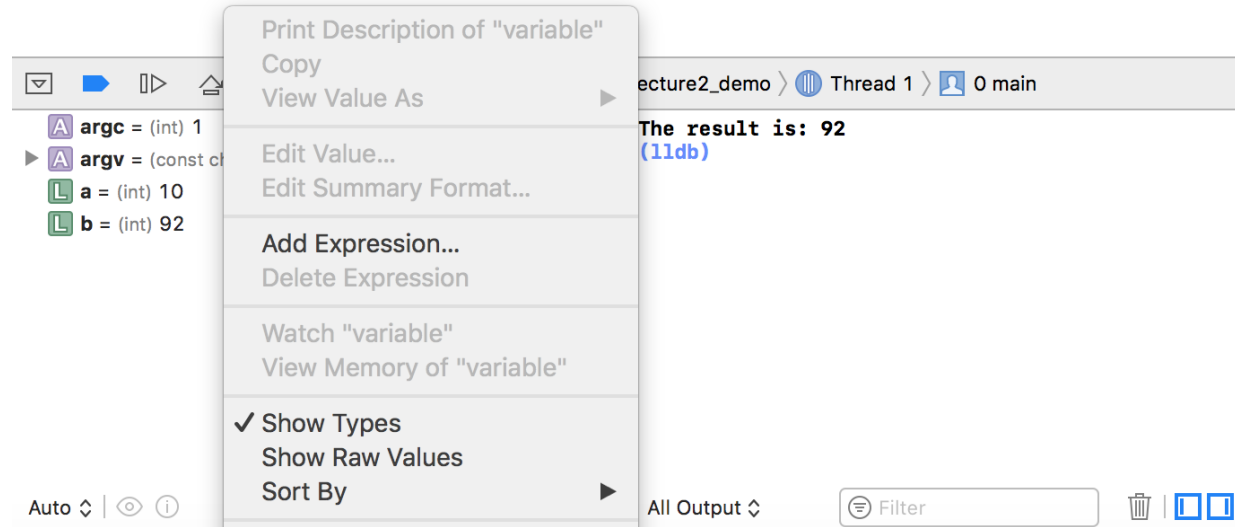
BTW, this window can also be used for accepting **user input**!



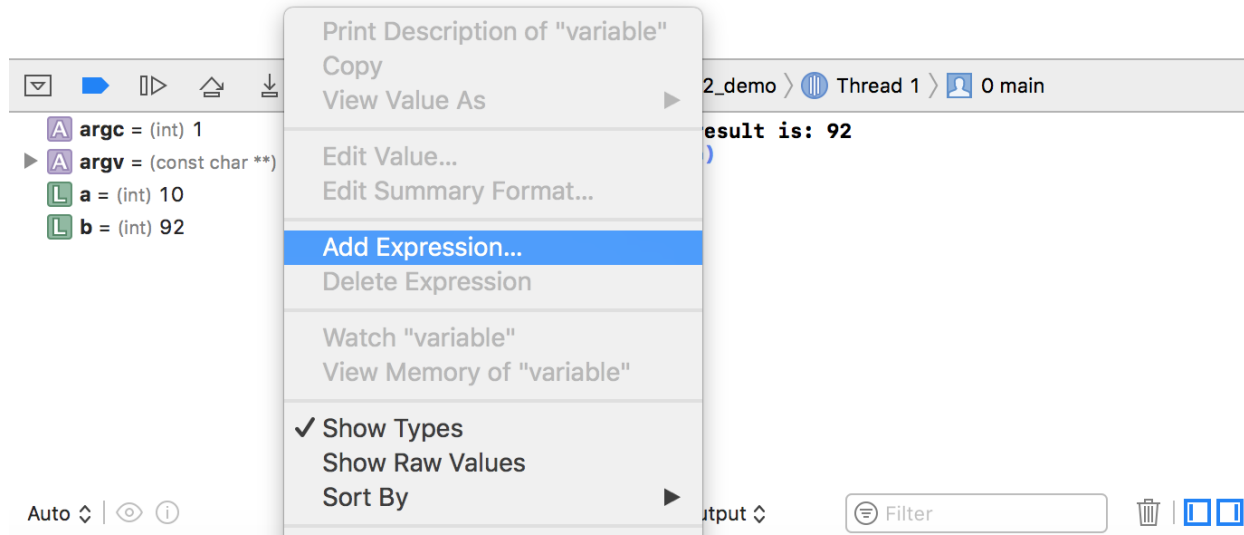
Debugging with Xcode

Let's use the **Watch** window to add an expression.

Right-Click
anywhere in the
Watch window.

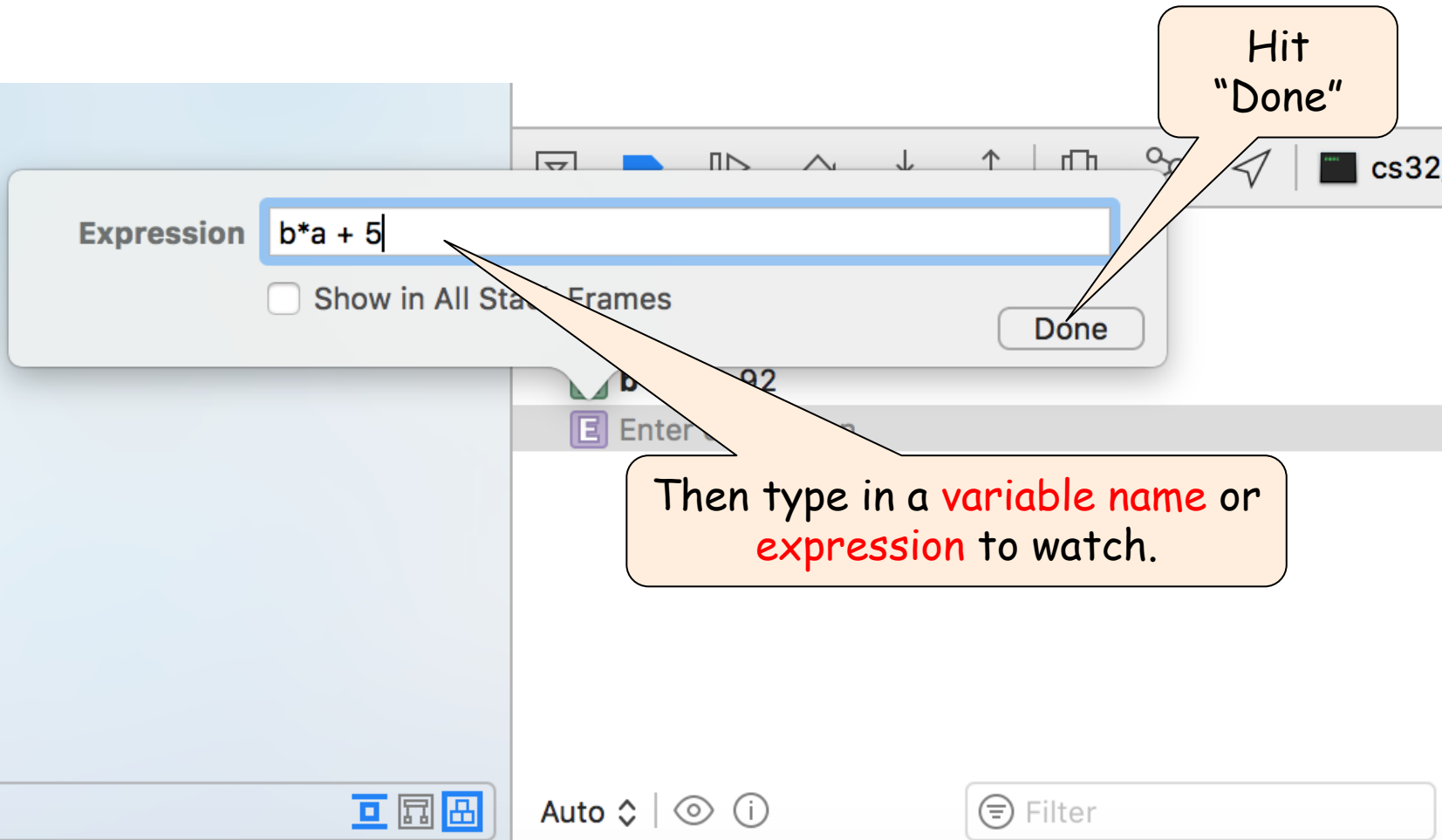


Now click on **Add
Expression...**



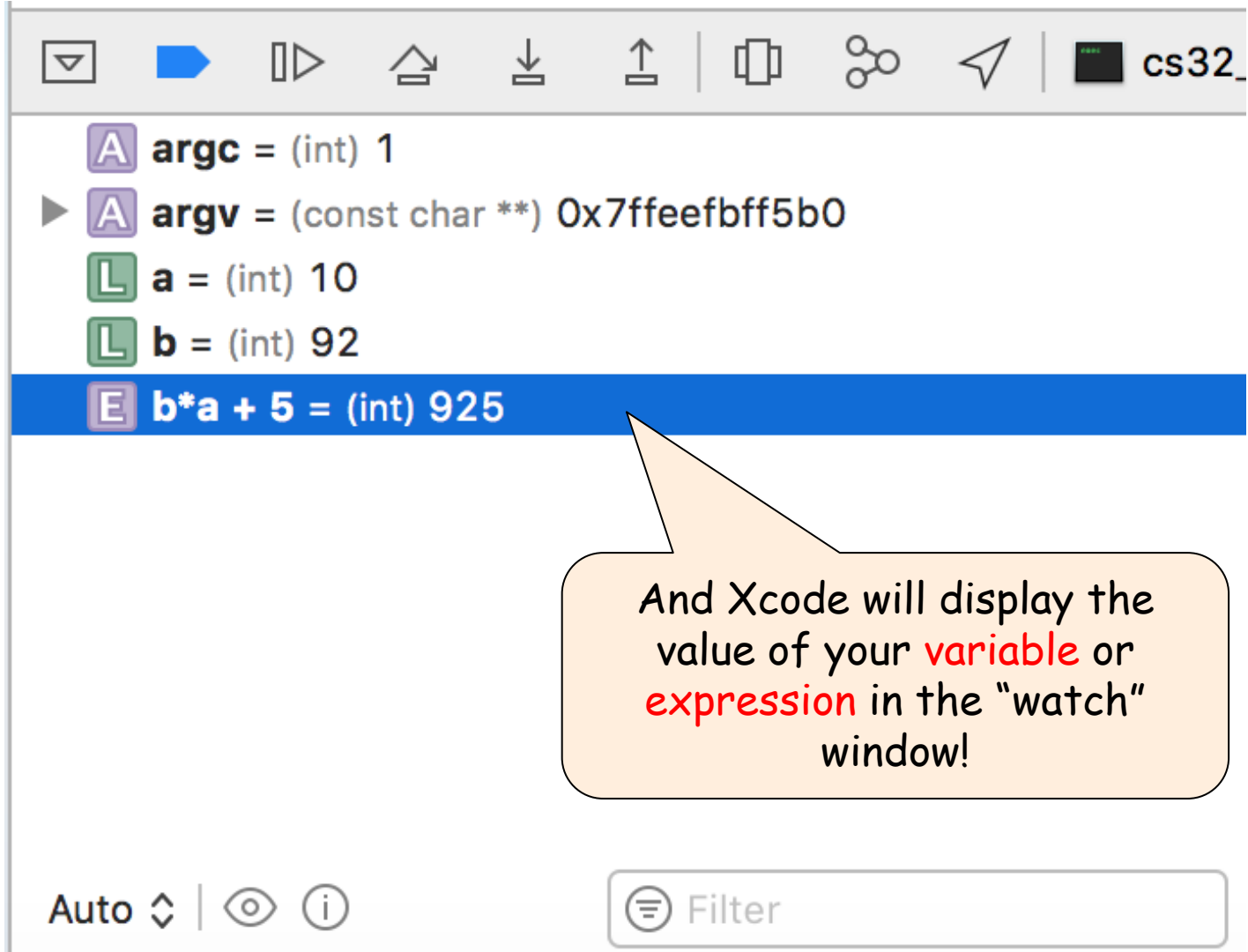
Debugging with Xcode

Add an expression to watch



Debugging with Xcode

Add an expression to watch



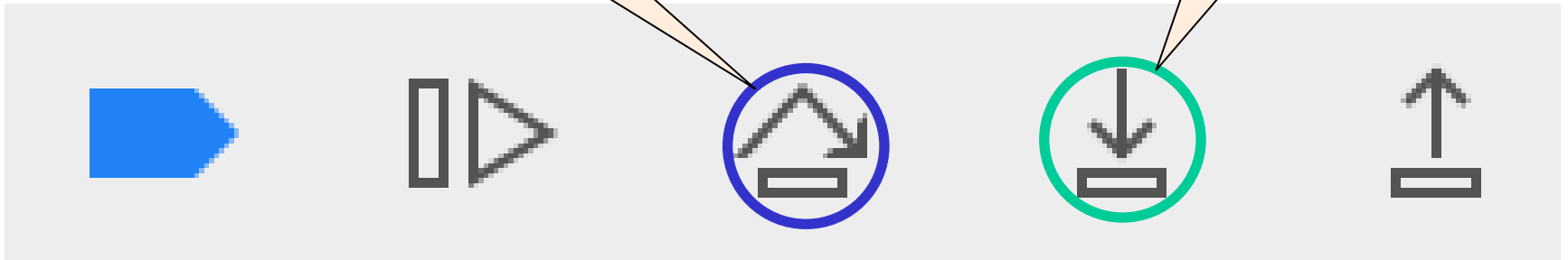
Debugging with Xcode

Let's revisit the "Step Over" button. Note the other button right next to it.

This is the "**Step Over**" button (the button we have been using so far).

"Step Over" and "Step Into" buttons do slightly different things. Let's see an example.

This is the "**Step Into**" button.



Debugging with Xcode

The screenshot displays the Xcode IDE interface for a C++ project named 'cs32_lecture2_demo'. The left sidebar shows system metrics (CPU, Memory, Energy Impact, Disk, Network) and a list of threads. The main editor window shows the source code of 'main.cpp'. The code defines a 'foo' function and a 'main' function. The 'main' function calls 'foo' and then prints the value of 'a'. The debugger is set to 'Step Over' mode, and a large green arrow points to the 'foo()' call site. The console shows the output of the program.

```
1 #include <iostream>
2 using namespace std;
3
4 void foo()
5 {
6     int c = 10;
7     cout << "c has a value of: " << c << endl;
8 }
9
10 int main(int argc, const char * argv[])
11 {
12     int a = 10;
13
14     foo();
15
16     cout << "a has a value of: " << a << endl;
17     return 0;
18 }
19
```

Thread 1 Queue: com.a...thread (serial)

0 main

- 1 start
- Thread 2
- Thread 3
- Thread 4

Thread 1: step over

cs32_lecture2_demo > Thread 1 > 0 main

args = (int) 1
argv = (const char **) 0x7ffeebf5b0
a = (int) 10

(lldb)

Auto | Filter | All Output | Filter

Debugging with Xcode

cs32_lecture2_demo > cs32_lecture2_demo > main.cpp > main(int argc, const char * argv[])

```
1 #include <iostream>
2 using namespace std;
3
4 void foo()
5 {
6     int c = 10;
7     cout << "c has a value of: " << c << endl;
8 }
9
10 int main(int argc, const char * argv[])
11 {
12     int a = 10;
13
14     foo();
15
16     cout << "a has a value of: " << a << endl;
17     return 0;
18 }
```

Notice that Xcode ran the entire `foo()` function from start to finish...

And now our cursor is on the line after the function call.

cs32_lecture2_demo > Thread 1 > 0 main

argc = (int) 1
argv = (const char **) 0x7ffeebf5b0
a = (int) 10

c has a value of: 10
(11db)

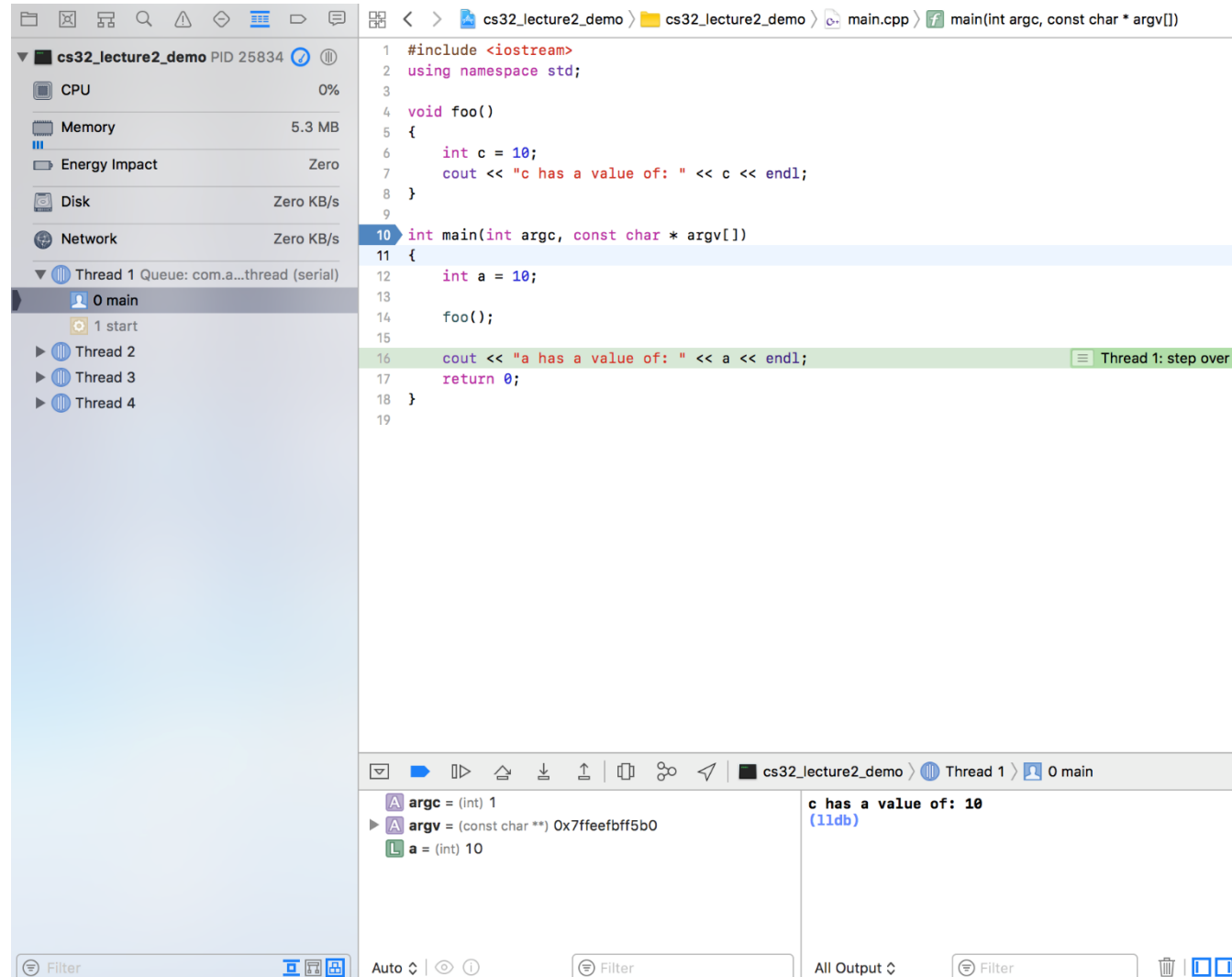
Auto ▾ | Filter | All Output ▾ | Filter

Debugging with Xcode

Summary:

If you hit "**Step Over**" while on a function call line, Xcode will run the entire function in one step. You won't be able to trace line-by-line through it.

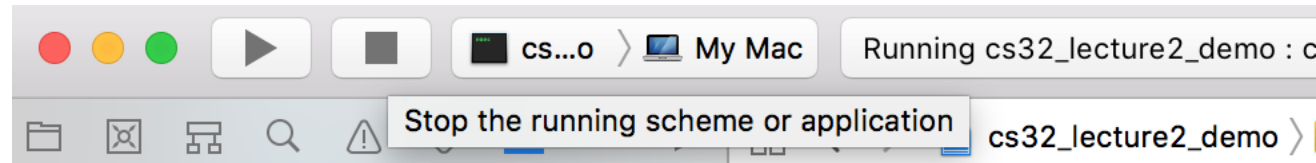
So "**Step Over**" lets you quickly run a line of code without delving into the details.



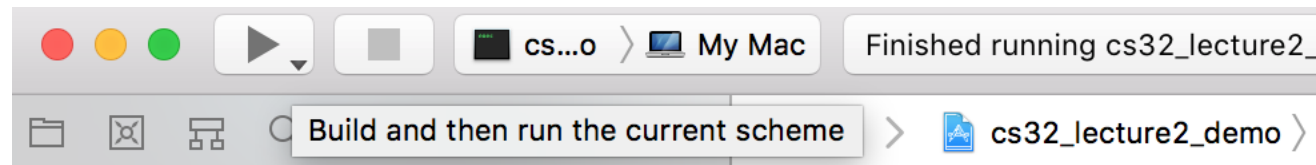
Debugging with Xcode

Let's restart the debugger and use the “**Step Into**” button instead of “**Step Over**”.

Just tap on the **Stop** button to stop the current debugging session.



Then tap on the **Run** button to re-build and run the code.



Debugging with Xcode

The screenshot shows the Xcode IDE with a C++ project named `cs32_lecture2_demo`. The code in `main.cpp` is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 void foo()
5 {
6     int c = 10;
7     cout << "c has a value of: " << c << endl;
8 }
9
10 int main(int argc, const char * argv[])
11 {
12     int a = 10;
13
14     foo();
15
16     cout << "a has a value of: " << a << endl;
17     return 0;
18 }
19
```

The debugger is set to `Thread 1: step over`. A large green arrow with the text "Step Into" points to the `Step Into` button in the debugger toolbar.

The debugger toolbar shows the following buttons: `Step Over`, `Step Into`, `Continue`, `Run`, `Stop`, `Breakpoint`, `Watch`, `Console`, and `Debugger`. The `Step Into` button is highlighted.

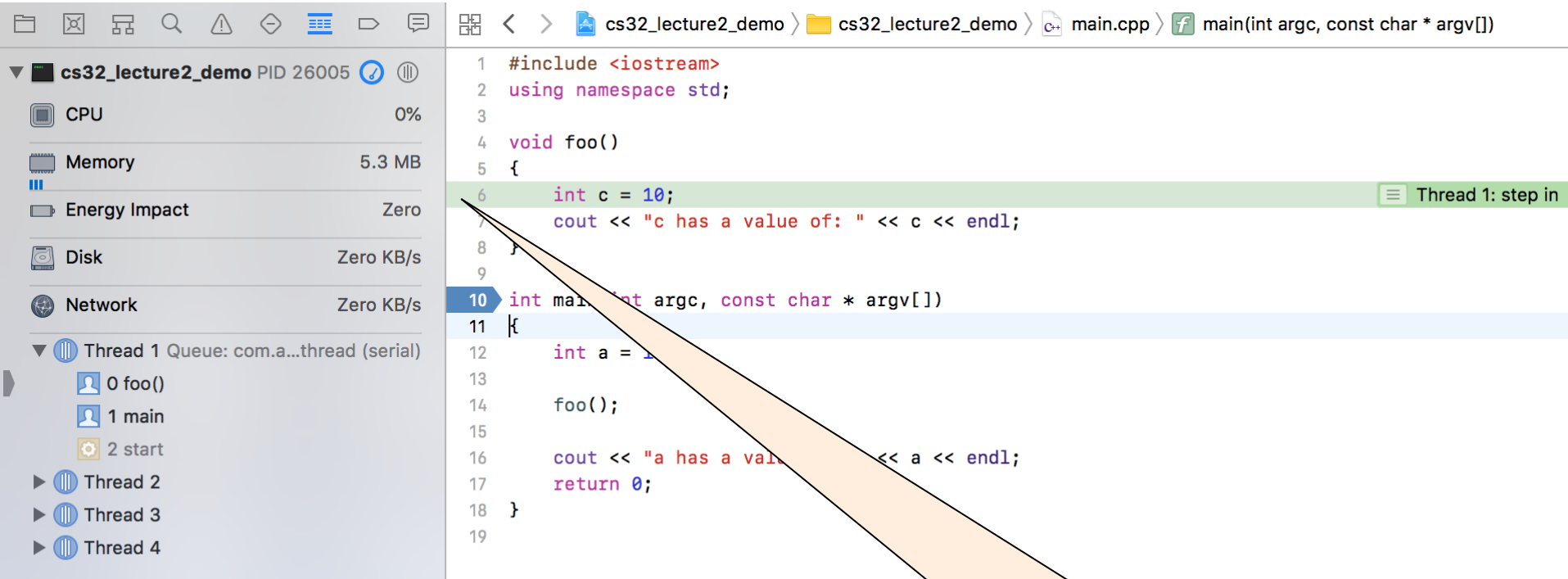
The `Variables` pane shows the following variables:

- `argc` = (int) 1
- `argv` = (const char **) 0x7ffeebf5b0
- `a` = (int) 10

The `Console` pane shows the following output:

```
(lldb)
>
a has a value of: 10
```

Debugging with Xcode



If you ever want to stop debugging line-by-line and just let your program run normally, then tap on this guy!

This is the **"Continue"** button.

Notice our debug cursor has moved into the `foo()` function. Now we trace through its code too (by hitting **"Step Over"** or **"Step Into"**)...

Debugging with Xcode

Quick note on breakpoints:

We already used breakpoints in our debugging examples, but there's 1 case where you really want to use them.

```
10
11 int main(int argc, const char * argv[])
12 {
13
14     foo();
15
16     for (int i=0; i<1000000; i++)
17     {
18         cout << "I'm wasting my time debugging a for loop" << endl;
19     }
20
21     double z = cos(10.0) / atan(20.0);
22
23     cout << "z has a value of: " << z << endl;
24     return 0;
25 }
```

Do I really need to step through this for loop?! That would take years!!!!

What if I want to start debugging at this line?

Debugging with Xcode



Debugging with Xcode

Just add a
breakpoint!

After pressing
Run, Xcode will
stop at that
line...

```
10
11 int main(int argc, const char * argv[])
12 {
13
14     foo();
15
16     for (int i=0; i<1000000; i++)
17     {
18         cout << "I'm wasting my time debugging a for loop" << endl;
19     }
20
21 double z = cos(10.0) / atan(20.0);
22
23     cout << "z has a value of: " << z << endl;
24     return 0;
25 }
26
```

After printing out a million of
these lines 😊

```
10
11 int main(int argc, const char * argv[])
12 {
13
14     foo();
15
16     for (int i=0; i<1000000; i++)
17     {
18         cout << "I'm wasting my time debugging a for loop" << endl;
19     }
20
21 double z = cos(10.0) / atan(20.0);
22
23     cout << "z has a value of: " << z << endl;
24     return 0;
25 }
26
```

Thread 1: breakpoint 1.1

Debugging

Learn how to use the debugger!

It can drastically speed up the
programming process!

Which means less
frustration



and more time
doing the things
you love!



Topic #1: Include Etiquette

A. Never include a CPP file in another .CPP or .H file.

file1.cpp

```
void someFunc()  
{  
    cout << "I'm cool!";  
}
```

You'll get linker errors.

Only include .H files within a .CPP file.

file2.cpp

```
#include "file1.cpp" // bad!  
  
void otherFunc()  
{  
    cout << "So am I!";  
    someFunc();  
}
```

Topic #1: Include Etiquette

B. **Never** put a “**using namespace**” command in a header file.

someHeader.h

```
#include <iostream>

using namespace std;
```

hello.cpp

```
#include "someHeader.h"

// BUT I DON'T WANT THAT
// NAMESPACE! YOU BASTARD!

int main()
{
    cout << "Hello world!";
}
```

So **this** is **bad**...

This is called
“Namespace Pollution”

Why? The .H file is
forcing CPP files that
include it to use its
namespace.

And that's just selfish.

Instead, just move the “using”
commands into your C++ files.

alcohol.h

```
class Alcohol
{
    ...
};
```

student.h

```
#include "alcohol.h"

class Student
{
    ...
private:
    Alcohol bottles[5];
};
```

main.cpp

```
#include "student.h"
#include "alcohol.h"
int main()
{
    Student larry;
    Alcohol vodka;

    cout << larry << " is drinking " << vodka;
}
```

Topic #1: Include?

And in fact, this works right now, because `student.h` does include `alcohol.h`.

Now your `main.cpp` file will compile correctly regardless of what's contained in the other header files it uses!

Always **DIRECTLY** include the header files you need **RIGHT** where you need them.

In this case, `main.cpp` has a `Student` variable and an `Alcohol` variable. So it should include both of their header files.

`alcohol.h` variable!

Topic #2: Preprocessor Directives

C++ has several special commands which you sometimes need in your programs.

Command 1: `#define`

You can use the `#define` command to define new constants:

`file.cpp`

```
#define PI 3.14159
#define FOOBAR

void someFunc()
{
    cout << PI;
}
```

You can also use `#define` to define a new constant **without specifying a value**! Why you ask? We'll see!

Preprocessor Directives

Command 2: `#ifdef` and `#endif`

You can use the `#ifdef` command to check if a constant has been defined already...

The compiler
only compiles the code
between the
`#ifdef` and the `#endif`
if the constant **was** defined.

Since FOOBAR was NOT defined I'll ignore the code until `#endif`.

file.cpp

```
/*  
void someFunc()  
{  
    cout << PI;  
}  
*/
```

Preprocessor Directives

Command 2: `#ifndef` and `#endif`

You can use the `#ifndef` command to NOT been defined already...

The compiler
only compiles the code
between the
`#ifndef` and the `#endif`
if the constant **was NOT** defined.

Since FOOBAR was
NOT defined I'll
compile the code
until `#endif`.

file.cpp

```
/*  
void someFunc()  
{  
    cout << PI;  
}  
*/
```

Separate Compilation

calc.h

```
class Calculator
{
public:
    int compute();
    ...
};
```

student.h

```
#include "calc.h"

class Student
{
public:
    void study();
    ...
private:
    Calculator myCalc;
};
```

Can anyone see
what the
problem is?

calc.cpp

```
#include "calc.h"
```

```
int Calculator::compute()
{
    ...
}
```

student.cpp

```
#include "student.h"

void Student::study()
{
    cout << myCalc.compute()
}
```

main.cpp

```
#include "student.h"
#include "calc.h"

int main()
{
    Student grace;
    Calculator hp;
    ...
    grace.study();
    hp.compute();
}
```

When using this composition (and
then you can use them like this in
bad) I happened to use both classes in
your main program
separate program files!

Separate Compilation

So what's the problem?

Well, since `main.cpp` and `student.h` both included `calc.h`, we ended up with **two definitions** of `Calculator`! That's bad!

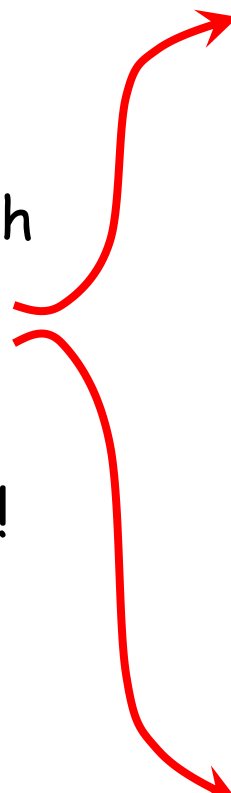
This can result in a compiler error!

So how do we fix it, you ask?

Here's how...

`main.cpp`

```
#include "student.h"
#include "calc.h"
int main()
{
    Student grace;
    Calculator hp;
    ...
    grace.study();
    hp.compute();
}
```



Separate C

main.cpp

Now, when the compiler compiles this code, it will ignore the redefined definitions!

Compiler:
STUDENT_H has been defined!
CALC_H has been defined!

```
class Calculator
{
public:
    void compute();
};
```

```
class Student
{
public:
    void study()
private:
    Calculator myCalc;
};
```

```
int main()
{
    Student grace;
    Calculator hp;
    ...
    grace.study();
    hp.compute();
}
```

Once the compilation is finished...

Compiler: The user has defined CALC_H. I'll ignore the code until the #endif!

And you can see that only one copy of each definition is now included!

Compiler: Ok, I just finished with the #ifndef CALC_H block of code...

Compiler: Ok, I just finished with the #ifndef CALC_H block of code...

I'll ignore the code until the #endif!

alcohol.h

```
class Alcohol
{
public:
    void drink() { cout << "glug!"; }
};
```

student.h

```
#include "alcohol.h"

class Student
{
public:
    void beIrresponsible();
    ...
private:
    Alcohol *myBottle;
};
```

So I need to include
`alcohol.h`, right?

I use the `Alcohol` class
(which is defined in `alcohol.h`)
to define a member variable

Last Topic: Knowing *WHEN* to Include .H Files

You might think that any
time you *refer to a class*,
you *should include its .h*
first... Right?

not so fast!

A.h

```
class FirstClass
{
public:
    void someFunc() { ... }
};
```

B.h

```
#include "A.h"

class SecondClass
{
public:
    void otherFunc()
    {
        FirstClass y;
        FirstClass b[10];
        y.someFunc();
        return(y);
    }

private:
    FirstClass x;
    FirstClass a[10];
};
```

Why? Because C++ needs to know the class's details in order to define actual variables with it or to let you call methods from it!

Last Topic: Knowing *WHEN* to Include .H Files

Here are the rules...

You must include the header file for any class (the full class)

... of that class's type, OR

2. You use the variable in any way (call a method on it, return it, etc).

On the other hand...

A.h

```
class FirstClass  
{  
public:  
    void someFunction();  
};
```

This line tells C++ that your class exists, but doesn't actually define all the gory details.

B.h

```
class FirstClass;  
  
class SecondClass  
{  
public:  
    void goober(FirstClass p1);  
    FirstClass hoover(int a);  
    void joober(FirstClass &p1);  
    void koober(FirstClass *p1);  
    void loober()  
    {  
        FirstClass *ptr;  
    }  
private:  
    FirstClass *ptr1, *z[10];  
};
```

Since none of this code actually uses the "guts" of the class (as the code did on the previous slide), this is all C++ needs to know!

Last Topic: Knowing When to Include .H Files

You do NONE of the work, but you...

define a function, OR

2. Use the class as the return type for a func, OR
3. Use the class to define a pointer or reference variable

Then you DON'T need to include the class's .H file.
(You may do so, but you don't need to)

Instead, all you need to do is give C++ a hint that your class exists (and is defined elsewhere).

Here's how you do that:

A.h

```
class FirstClass
{
public: // thousands of lines
    ... void someFunc() { lines }
};
```

B.h

```
#include "A.h" // really slow!

class SecondClass
{
public:

private:

};
```

Last Topic: Knowing *WHEN* to Include .H Files

Wow - so confusing!

Why not just always use `#include` to avoid the confusion?

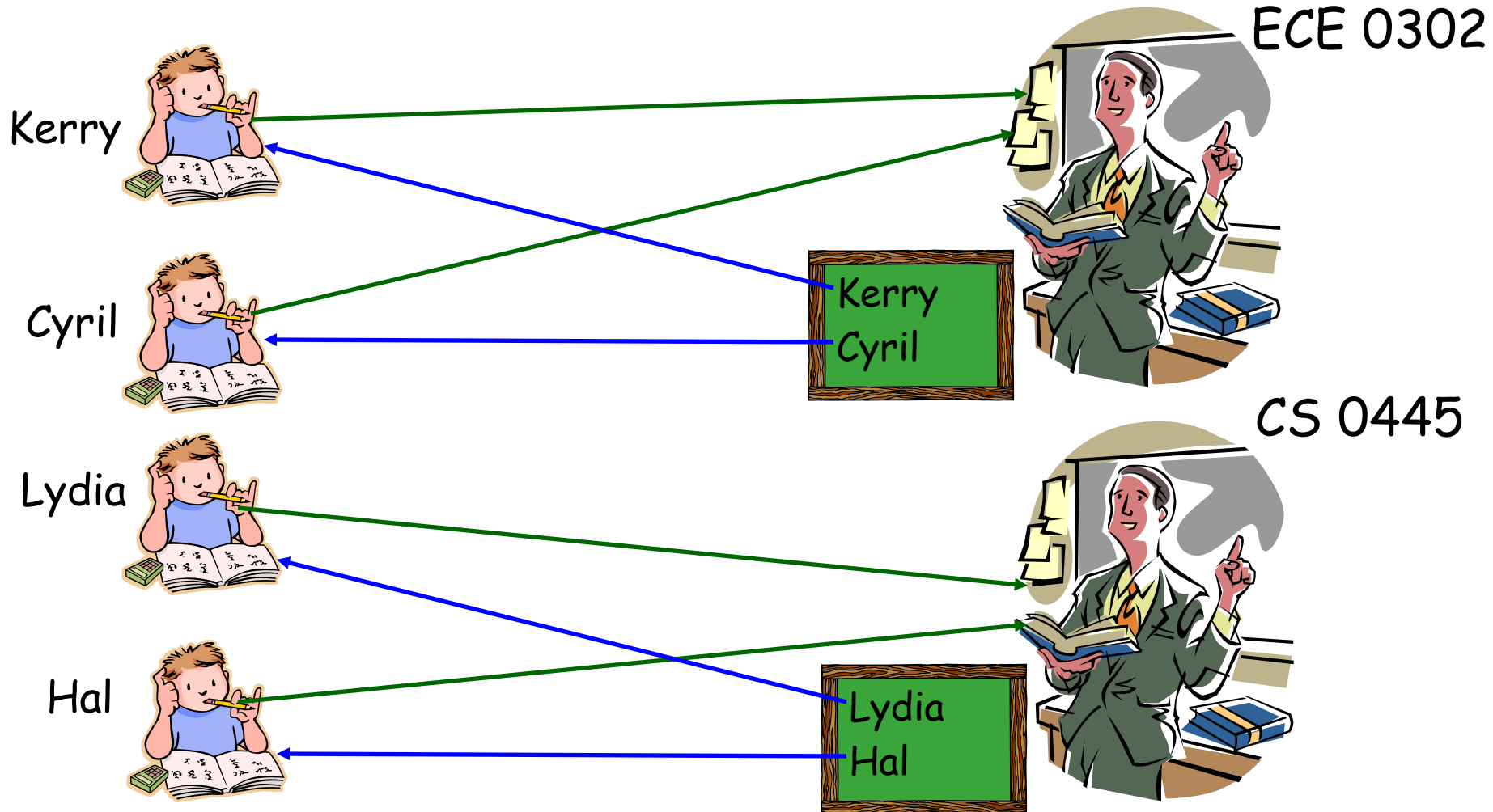
There are two reasons:

1. If a .h file is large (thousands of lines), `#including` it when you don't strictly need to slows down your compilation!
2. If two classes refer to each other, and both classes try to `#include` the other's .H file, you'll get a compile error.

Students and Classrooms

Every student knows what class he/she is in...

Every class has a roster of its students...



These type of **cyclical relationships** cause #include problems!

Last Topic: Self-referential Classes

And here we have the `Student` class. Note that `Student` knows which `ClassRoom` it is in...
Here we have a `ClassRoom` which includes a bunch of `Students`...

Student.h

```
#include "ClassRoom.h"

class Student
{
public:
    ...

private:
    ClassRoom *m_myRoom;
};
```



ClassRoom.h

```
#include "Student.h"

class ClassRoom
{
public:
    ...

private:
    Student m_studs[100];
};
```



Student.cpp

```
#include "Student.h"

void Student::printMyClassRoom()
{
    cout << "I'm in Boelter #" <<
        m_myRoom->getRmNum();
}
```

ClassRoom.cpp

```
#include "ClassRoom.h"

void ClassRoom::printRoster()
{
    for (int i=0; i<100; i++)
        cout << m_studs[i].getName();
}
```

Last Topic: Self-referential Classes

main.cpp

```
#include "Student.h"
```

Student.h

```
#include "ClassRoom.h"
```

The compiler will keep
going forever!!!!

(oh, and adding Include Guards
doesn't solve the underlying
problem either!)

```
public:
```

```
...
```

```
private:
```

```
    Student m_studs[100];
```

```
};
```

So how do we solve this cyclical nightmare?

Step #1:

Look at the two class definitions in your .h files. At least one of them should NOT need the full #include.

Why does this class NOT need the full #include? Why?

This class **JUST** defines a **pointer to a Classroom**. It does NOT hold a full Classroom variable and therefore doesn't require the full class definition here!!!

This class defines actual **Student variables**... It therefore requires the full class definition to work!

Student.h

```
#include "ClassRoom.h"

class Student
{
public:
    ...

private:
    Classroom *m_myRoom;
};
```

ClassRoom.h

```
#include "Student.h"

class Classroom
{
public:
    ...

private:
    Student m_studs[100];
};
```

So how do we solve this cyclical nightmare?

Step #2:

Take the class that does NOT require the full #include (forget the other one) and update its header file:

Replace the **#include "XXXX.h"** statement
with the following: **class YYY;**

Where **YYY** is the other class name (e.g., **ClassRoom**).

Student's

```
class Class
```

```
class st
```

```
{  
public:
```

```
...
```

```
private:
```

```
    ClassRoom *m_myRoom , haha;
```

```
};
```

The Programmer says:

I think I have a little for
see what the compiler says
define a full variable

The Compiler says:

The Compiler replies:

"Wanna play nasty? Ok.
Here are 9000 **SYNTAX ERRORS**
for you to deal with!"

variables with

or call any member functions

```
};
```

So how do we solve this cyclical nightmare?

All you need is this **class declaration**;
line to give the compiler
a heads-up.

Step #3:

Almost done. Now update the
CPP file for this class by
including the other header
file.

Since this .H file JUST has a
pointer to a Classroom (but no code
in this file that uses or defines a full
ClassRoom variable)...

compiler:

So

header here!

my CPP file

is going to actually
ClassRoom's functionality.

Student.h

```
class Classroom;
```

Student.cpp

```
#include "Student.h"  
#include "ClassRoom.h"  
void Student::printMyRoom()  
{  
    cout << "I'm in Boelter #" <<  
    m_myRoom->getRmNum();  
}
```

we're calling a member
function of Classroom
here...

The Compiler says:

Now, now I can see the full
definition of what a Classroom
variable really is...
The compiler is happy as
long as you #include the
class definition before you
actually use the class in
your functions...

So how do we solve this cyclical nightmare?

Student.h

```
class Classroom;  
  
class Student  
{  
public:  
    void beObnoxious();  
    cout << m_myRoom->getRmNum() << " sucks!"; }  
private:  
    Classroom *m_myRoom;  
};
```

The Compiler says:

Hey! Wait a second, you're calling the getRmNum() function but you haven't defined the full class.

No WAY!
Fix it or DIE!

#4:

Make sure that your code doesn't have any other violations that violate C++ class rules...

Student.cpp

```
#include "Student.h"  
#include "ClassRoom.h"  
  
void Student::beObnoxious() {  
    cout << m_myRoom->getRmNum() << " sucks!"; }
```

The Compiler says:

Alright. That's better.

Don't let it happen again.

Make sure you have defined one or more functions DIRECTLY in the .cpp file AND they are part of the class in a header file. When you must include a header file, MOVE them to the CPP file.

So how do we solve this cyclical nightmare?

Now let's look at both of our classes... Notice, we no longer have a cyclical reference! Woohoo!

Student.h

```
class Classroom;

class Student
{
public:
    ...

private:
    Classroom *m_myRoom;
};
```

ClassRoom.h

```
#include "Student.h"

class Classroom
{
public:
    ...

private:
    Student m_studs[100];
};
```

Student.cpp

```
#include "Student.h"
#include "ClassRoom.h"

void Student::printMyClassRoom()
{
    cout << "I'm in Boelter #" <<
        m_myRoom->getRmNum();
}
```

ClassRoom.cpp

```
#include "ClassRoom.h"

void Classroom::printRoster()
{
    for (int i=0; i<100; i++)
        cout << m_studs[i].getName();
}
```

Default Arguments!

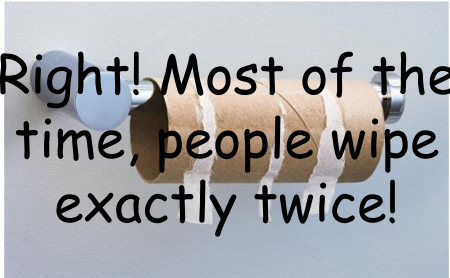
```
void poop(string name, int numberOfWipes)
{
    cout << name << " just pooped.\n";
    cout << "They wiped " << numberOfWipes <<
        " times.\n";
}

...

int main()
{
    poop("Phyllis", 2);
    poop("Astro", 2);
    poop("Sylvia", 2);
    poop("Carey", 3);
    poop("David", 0);
    poop("Sergey", 2);
    poop("Larry", 2);
    poop("Devan", 2);
}
```

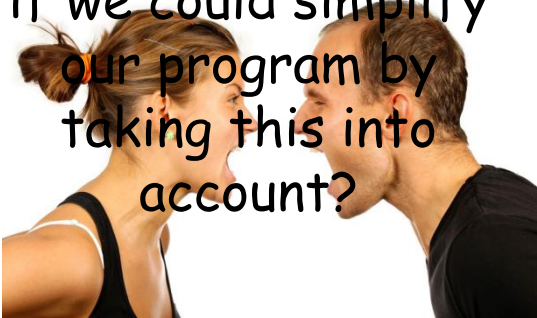
So what do you notice about the following code?

(other than David really needs to wipe)



Right! Most of the time, people wipe exactly twice!

Wouldn't it be nice if we could simplify our program by taking this into account?



Default Arguments!

```
void poop(string name, int numberOfWipes) = 2
{
    cout << name << " just pooped.\n";
    cout << "They wiped " << numberOfWipes <<
        " times.\n";
}
```

...

```
int main()
{
```

```
→ poop("Phyllis" ) // defaults to passing in 2
```

```
    poop("Astro" );
```

```
    poop("Sylvia" );
```

```
    poop("Carey", 3);
```

```
→ poop("David", 0); // ignores the default!
```

```
    poop("Sergey" );
```

```
    poop("Larry" );
```

```
    poop("Devan" );
```

```
}
```

We can!
Let's see how!

C++ lets you specify
a default value for
a parameter...
like this...

Now, when you call
the function, you can
leave out the
parameter if you just
want the default...

and C++ will
automagically pass
in that default
value!

Note: We still need to
pass in the value any time
we don't want the default!

Default Arguments!

```
void fart(int length 10, int volume 50)
{
    cout << "I just farted for " << length
        << " seconds, and it was"
        << volume << " decibels loud.\n"
}

int main()
{
    →fart(20,5);      // long but not so loud
    →fart(5);         // short violent burst!
    →fart();          // medium and pretty loud
    →fart(,30);       // NOT ALLOWED
}
```

You can have more than one default parameter if you like, and C++ will figure out what to do!

But you can't do something like this!

Why not? Good question - ask the C++ language designers. ☺



Default Arguments: One other thing

If the j^{th} parameter has a default value, then all of the following parameters ($j+1$ to n) **must** also have default values.

```
// INVALID! Our 1st param is default, but 2nd and 3rd aren't!  
bool burp(int length = 5, int loudness, int pitch)
```

```
// INVALID! Our first 2 params are default, but 3rd isn't!  
bool burp(int length = 5, int loudness = 12, int pitch)
```

```
// INVALID! Our 1st param is default, but 2nd isn't!  
bool burp(int length = 5, int loudness, int pitch = 60)  
{  
    ...  
}
```

```
// PERFECT! All parameters are default  
bool burp(int length = 5, int loudness = 5, int pitch = 60)
```

```
// PERFECT! All default parameters come after non-defaults  
bool burp(int length, int loudness = 5, int pitch = 60)  
{  
    ...  
}
```

Challenge #1

Write a class called `Quadratic` which represents a second-order quadratic equation, e.g.: $4x^2+3x+5$

When you `construct` a `Quadratic` object, you pass in the three coefficients (e.g., `4`, `3`, and `5`) for the equation.

The class also has an `evaluate` method which allows you pass in a value for `x`. The function will return the value of $f(x)$.

The class has a `destructor` which prints out `"goodbye"`.

Challenge #2

Write a class called `MathNerd` which represents a math nerd. Every `MathNerd` has his own special quadratic equation!

When you `construct` a `MathNerd`, he always wants you to specify the first two coefficients (for the x^2 and x) for his equation.

The `MathNerd` has a `getMyValue` function which accepts a value for x and should return $f(x)$ for the nerd's QE.